

《数据结构》课程设计总结

学号： 2154312

姓名： 郑博远

专业： 计算机科学与技术

2023 年 7 月

目 录

第一部分 算法实现设计说明.....	1
1.1 题目	1
1.2 软件功能.....	1
1.3 设计思想.....	2
1.4 逻辑结构与物理结构.....	6
1.5 开发平台.....	8
1.6 系统的运行结果分析说明.....	9
1.7 操作说明.....	10
第二部分 综合应用设计说明.....	26
2.1 题目	26
2.2 软件功能.....	26
2.3 设计思想.....	29
2.4 逻辑结构与物理结构.....	30
2.5 开发平台.....	34
2.6 系统的运行结果分析说明.....	35
2.7 操作说明.....	42
第三部分 实践总结.....	54
3.1.所做的工作.....	54
3.2.总结与收获.....	55
第四部分 参考文献.....	56

第一部分 算法实现设计说明

1.1 题目

二叉树，完成：

- (1) 建立一棵二叉树,并对它进行先序、中序、后序遍历；
- (2) 统计树中的叶子结点个数；
- (3) 分别对它进行先序、中序、后序线索化；
- (4) 实现先序、中序线索树的遍历；
- (5) 显示该树和线索化后的树（此要求可视情况选择是否完成）。

1.2 软件功能

1.2.1 二叉树的建立与叶子结点个数统计

• 功能描述：

用户可以通过在画布上点击鼠标创建二叉树的结点，以及通过鼠标拖动连接结点来建立父子关系。

• 图形化界面说明：

1. 用户启动程序后，会看到一个空的画布，提示用户点击画布来创建根节点。
2. 用户点击画布的某位置后，一个带有编号的圆形结点会在该位置出现，表示根节点。
3. 用户可以点击已有结点，并通过拖动鼠标创建其子结点（鼠标左键单击对应左子结点，鼠标右键单击对应右子结点），从而构建整棵二叉树。
4. 叶子结点的个数实时统计，并显示在窗口右侧。

• 实现方式：

1. 创建一个 `graphicsView` 类作为画布，继承自 `QGraphicsView` 类，用户可以通过在画布上点击鼠标来创建二叉树的结点。

2. 创建一个 `graphicsVexItem` 类来表示二叉树的结点，继承自 `QGraphicsEllipseItem` 类，通过该类来绘制结点的圆形图形。

3. 使用二叉树的数据结构来保存用户创建的结点，每个结点对象会在界面上显示，并在内部数据结构中建立对应的树结构。

4. 使用鼠标事件捕捉用户的点击拖动操作，根据鼠标操作来创建结点和建立父子关系。

1.2.2 二叉树的遍历

• 功能描述：

实现了先序、中序、后序遍历的递归算法，通过递归访问二叉树的各个结点，将遍历结果存储并返回。

• 图形化界面说明：

1. 用户可以在界面上选择遍历方式，如先序、中序或后序。
2. 用户点击“开始遍历”按钮后，程序会开始遍历二叉树，并将遍历的结点高亮显示，用户可以观察遍历的过程。
3. 遍历结果将以动画形式在界面上显示，用户可以清晰地看到遍历的顺序和遍历结果。

• 实现方式：

1. 在二叉树的数据结构中实现先序、中序、后序遍历的算法。本次课程设计中采用非递归方式实现先序、中序与后序遍历的算法^[1]。
2. 用户选择遍历方式后，调用相应的遍历函数，遍历过程中通过信号槽机制将遍历的结点高亮显示。
3. 遍历的结果会以动画形式在界面上显示，用户能清晰地看到遍历的顺序和遍历结果。

1.2.3 二叉树的线索化

• 功能描述：

实现了将二叉树线索化为先序线索二叉树、中序线索二叉树以及后序线索二叉树的算法，便于后续的线索树遍历。

- **图形化界面说明:**

1. 用户可以选择先序线索化、中序线索化或后序线索化，并点击相应按钮执行操作。
2. 执行线索化后，用户可以在界面上看到线索化后的二叉树结构，线索化的结点通过虚线标出线索体现逻辑关系。
3. 对于先序、中序线索化，程序自动继续执行线索化树的遍历，以便观察线索树遍历的效果，遍历结果也将以动画形式在界面上显示。

- **实现方式:**

1. 通过遍历二叉树的节点，在遍历过程中对每个节点进行线索化处理。对于每个节点，判断其左子树和右子树是否为空，若为空，则将其左子树指向其前驱节点，将其右子树指向其后继节点，以建立线索关系。
2. 采用递归或迭代的方式进行遍历，根据遍历顺序完成线索化操作^{[2][3]}。可以选择先序线索化、中序线索化或其他线索化方式，用户可以在图形化界面上选择线索化方式。
3. 实现线索化算法后，通过图形化界面展示线索化的结果^[4]。线索化后的节点用不同的颜色与虚线标记表示，以便用户清晰地观察线索化后的二叉树结构。

1.3 设计思想

本次程序设计主要分为三个部分。首先，是算法的核心设计与实现，包括满足题目要求的所有内部算法和数据结构（即二叉树结构以及对应的遍历与线索化算法），但不包含任何与图形界面相关的内容。其次，是关于图形显示的设计，专门负责处理界面的呈现，而不涉及具体的算法。最后，将这两个部分整合在一起，实现图形界面与内部数据结构之间的协同工作。在进行本次程序开发中，我遵循了 MVC（Model View Controller）模式的设计思路。在 MVC 模式中，Model 代表数据和核心逻辑，View 代表界面显示，Controller 负责控制数据与界面之间的交互。

1.3.1 模型层 (Model)

模型层负责管理二叉树的数据和逻辑操作。在代码中，模型层由以下部分组成：

- **binaryTreeNode 类:**

这是二叉树结点的抽象类，定义了二叉树结点的基本属性和方法。通过纯虚函数^[5]，为不同类型的二叉树结点提供了一致的接口。binaryTree 类能够通过调用以下虚函数实现遍历

操作，而无需关心可视化的数据结构实现细节：

1. `getLeftChild()` 和 `getRightChild()` 方法

```
virtual binaryTreeNode* getLeftChild() const = 0;  
virtual binaryTreeNode* getRightChild() const = 0;
```

这两个虚方法分别返回当前节点的左孩子和右孩子节点。它们允许 `binaryTree` 类通过递归访问二叉树的不同部分。

2. `getLeftChildTag()` 和 `getRightChildTag()` 方法：

```
virtual enum TAG getLeftChildTag() const = 0;  
virtual enum TAG getRightChildTag() const = 0;
```

这两个虚方法返回左孩子和右孩子的标志（LINK 或 THREAD），用于判断当前节点的孩子是实际子节点还是线索。

3. `setLeftChild()` 和 `setRightChild()` 方法：

```
virtual void setLeftChild(binaryTreeNode* leftChild, enum  
binaryTreeNode::TAG tag) = 0;  
virtual void setRightChild(binaryTreeNode* rightChild, enum  
binaryTreeNode::TAG tag) = 0;
```

这两个虚方法允许 `binaryTree` 类设置当前节点的左孩子和右孩子的指针值以及标签。

4. `visit()` 方法：

```
virtual void visit() = 0;
```

这是一个虚方法，用于定义对当前节点的访问操作。在实际的二叉树遍历中，`binaryTree` 类可以根据遍历顺序执行不同的操作。

- **`binaryTree` 类：**

这是二叉树的类，负责管理整棵树的结构和操作。它包含了根节点、前序、中序、后序遍历方法、线索化方法等。这些方法通过递归与非递归的方式实现了对二叉树的各种操作。

- **模型层的设计思想：**

模型层通过分层抽象将二叉树的结点和树的操作进行了分层抽象,使得可以灵活地处理不同类型的二叉树。同时,其使用合适的数据结构来表示二叉树,通过节点之间的连接关系来构建树的结构。通过封装了与二叉树相关的逻辑,包括遍历、线索化等操作,使得这些操作可以在不同的视图和控制器中重复使用。

1.3.3 视图层 (View)

视图层负责用户界面的显示和交互。在代码中,视图层由以下部分组成:

- **graphicsView 类:**

这是二叉树的可视化画布,继承自 `QGraphicsView`。它负责绘制二叉树的结点、边和线索,以及处理用户的鼠标事件(如单击空白画布创建根节点)。

- **graphicsVexItem 类:**

这是二叉树结点的可视化表示,继承自 `QGraphicsEllipseItem` 和 `binaryTreeNode`。它负责绘制结点的形状和颜色,并处理与用户的交互,如点击拖拽子结点事件。

- **graphicsEdgeItem 类:**

这是二叉树边的可视化表示,继承自 `QGraphicsLineItem`。它负责绘制二叉树的边。

- **graphicsThreadItem 类:**

这是二叉树线索的可视化表示,继承自 `QGraphicsItem`。它负责绘制线索。

- **视图层的设计思想:**

视图层通过图形界面显示二叉树,提供了交互式的操作方式,包括创建结点、删除结点、遍历等。通过捕获点击拖动等鼠标事件以响应用户的操作,并通过信号与控制器通信。同时,视图层负责将模型层的数据可视化展示,通过不同颜色和形状区分结点、边、线索。

1.3.3 控制器层 (Controller)

控制器层是模型和视图之间的桥梁,负责协调用户输入和模型操作。首先,控制器层使用 Qt 中的类对模型进行再封装,创建必要的接口以供视图层调用,从而实现了模型的抽象。这样,视图层不需要了解模型的具体实现细节,只需通过控制器提供的接口与模型进行交互。如使用 `binaryTree` 类中的遍历方法,通过控制器层接收用户的遍历命令,并调用模型层的

相应方法来执行遍历操作。

其次，控制器层负责维护模型与视图的同步。由于采用了 `binaryTreeNode` 虚基类继承的方式建立可视化结点，不需要额外进行两次数据的存储，因此数据能保持自然的同步。这种同步保证了模型和视图的一致性，使得用户操作能够正确反映在界面和数据之间。

1.4 逻辑结构与物理结构

1.4.1 逻辑结构

由于本次课程设计的算法实现部分对应的是二叉树的遍历，因此逻辑结构对应的是处理与二叉树的建立、遍历、线索化等操作相关的类。这些操作基于树型结构这一种非线性数据结构。二叉树（Binary Tree）是一种树型结构，它的特点是每个结点至多只有两棵子树，并且子树有左右之分，其次序不能任意颠倒。

1. `binaryTreeNode` 类（抽象类）

- 该类是所有二叉树节点的基类，用于定义节点的通用接口。
- 强调了树型结构的逻辑关系，包括获取左右子节点、设置子节点、节点访问等操作。
- 通过虚函数实现多态性，使不同类型的二叉树节点都能继承这一通用接口。

```
class binaryTreeNode
{
    // ...其他成员...
public:
    virtual binaryTreeNode* getLeftChild() const = 0;
    virtual binaryTreeNode* getRightChild() const = 0;
    virtual enum TAG getLeftChildTag() const = 0;
    virtual enum TAG getRightChildTag() const = 0;
    virtual void setLeftChild(binaryTreeNode* leftChild, enum
binaryTreeNode::TAG tag) = 0;
    virtual void setRightChild(binaryTreeNode* rightChild, enum
binaryTreeNode::TAG tag) = 0;
    virtual void visit() = 0;
    virtual ~binaryTreeNode() = 0;
};
```

2. `binaryTree` 类

- 该类实现了各种二叉树遍历和线索化的算法，是逻辑结构的核心。
- 在逻辑层面，它强调了遍历、线索化等树型数据结构的操作，但不涉及具体的可视化表示。

```
class binaryTree
{
    // ...其他成员...
public:
    qint16 countLeafNode();
    void preOrderTraversal(bool withDelay = true);
    void inOrderTraversal(bool withDelay = true);
    void postOrderTraversal(bool withDelay = true);
    void preOrderTraversal_Thr(bool withDelay = true);
    void inOrderTraversal_Thr(bool withDelay = true);
    void createThreadedTree(int mode, bool withDelay = true);
    void clearThreadedTree();

    // ...其他成员...
};
```

1.4.2 物理结构

在二叉树的存储中，常用的物理结构有顺序存储结构与链式存储结构。

- 链式存储（Linked Storage）：

链式存储是树形结构的一种常见方式，它使用节点之间的引用或指针来构建树。每个节点包含了数据以及指向其子节点的指针。这种存储方式适用于不规则的树结构，其中每个节点的子节点数量可以不同。链式存储通常由面向对象的语言中的类或结构体表示，其中每个节点对象包含数据和指向左右子节点的引用。这种存储方式在二叉树中非常常见，因为它使得遍历和修改树的操作更加方便。二叉树的链式存储包括了二叉树节点类，每个节点包含数据、左子节点指针和右子节点指针。

- 顺序存储（Sequential Storage）：

顺序存储是一种使用数组或列表来表示树结构的方式。通常，这种存储方式用于完全二叉树，其中每个节点都按照从左到右的顺序排列，没有空缺。通过数组的索引关系，可以轻松找到节点之间的父子关系。例如，对于节点 i ，其左子节点位于 $2i+1$ 的位置，右子节点位于 $2i+2$ 的位置，父节点位于 $(i-1)/2$ 的位置。顺序存储通常用于堆（如二叉堆）等数据结构

的实现，其中快速查找和调整节点的位置是关键操作。这种存储方式的优点是节省空间，因为不需要存储额外的指针。但在非完全二叉树或动态树结构中使用起来更为复杂。

本次程序采用了顺序存储的存储结构，具体体现在二叉树的可视化结点类 `graphicsVecItem` 中。每个结点有左右指针指向其左右孩子结点，并有两个 `tag` 来标记其是否是线索。

```
// 二叉树的可视化结点
class graphicsVexItem: public QObject, public QGraphicsEllipseItem, public
binaryTreeNode
{
    // ...其他成员...

private:
    // 左右孩子及 tag
    graphicsVexItem* leftChild = nullptr, * rightChild = nullptr;
    enum binaryTreeNode::TAG leftChildTag = binaryTreeNode::LINK;
    enum binaryTreeNode::TAG rightChildTag = binaryTreeNode::LINK;

public:
    // 对基类虚方法的继承
    virtual graphicsVexItem* getLeftChild() const Q_DECL_OVERRIDE;
    virtual graphicsVexItem* getRightChild() const Q_DECL_OVERRIDE;
    virtual enum TAG getLeftChildTag() const Q_DECL_OVERRIDE;
    virtual enum TAG getRightChildTag() const Q_DECL_OVERRIDE;
    virtual void setLeftChild(binaryTreeNode* leftChild, enum
binaryTreeNode::TAG tag) Q_DECL_OVERRIDE;
    virtual void setRightChild(binaryTreeNode* rightChild, enum
binaryTreeNode::TAG tag) Q_DECL_OVERRIDE;
    virtual void visit() Q_DECL_OVERRIDE;

    // ...其他成员...
};
```

1.5 开发平台

计算机型号：联想小新 Pro 14ACH 2021

计算机内存：16.00GB

CPU：AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz

操作系统：Windows 10 家庭版

开发语言：C++（C++11 标准以上）

开发框架：QT

集成开发环境：Qt Version 5.12.12

编译器：MinGW 7.3.0 64-bit

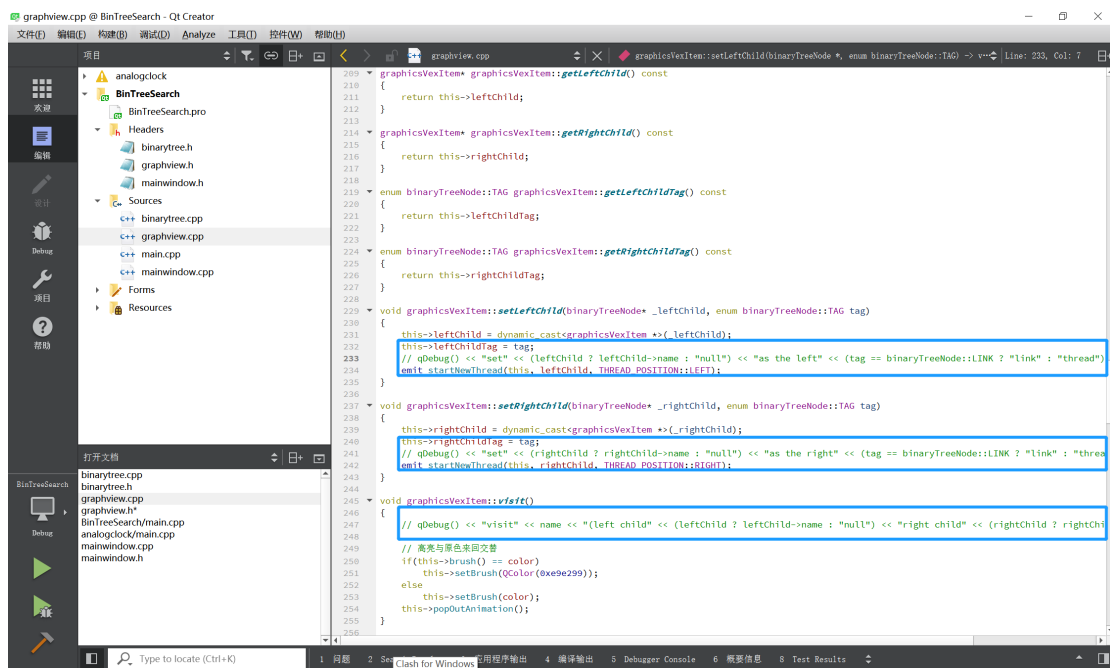
运行环境：通过 windeployqt.exe 及 Enigma Virtual Box 进行整合压缩为可执行文件，可在普通 windows 机型下运行。

1.6 系统的运行结果分析说明

1.6.1 调试与开发过程

1. 通过 qDebug 进行输出

在开发过程中，使用了 Qt 框架提供的 QDebug 输出来实时输出调试信息。这些输出信息包括了程序的执行流程、变量的值、以及函数的调用。通过 QDebug 输出，可以快速定位和解决潜在的问题，如内存泄漏、逻辑错误等。这些输出信息有助于我在开发过程中跟踪代码的执行路径，并确认算法的正确性。

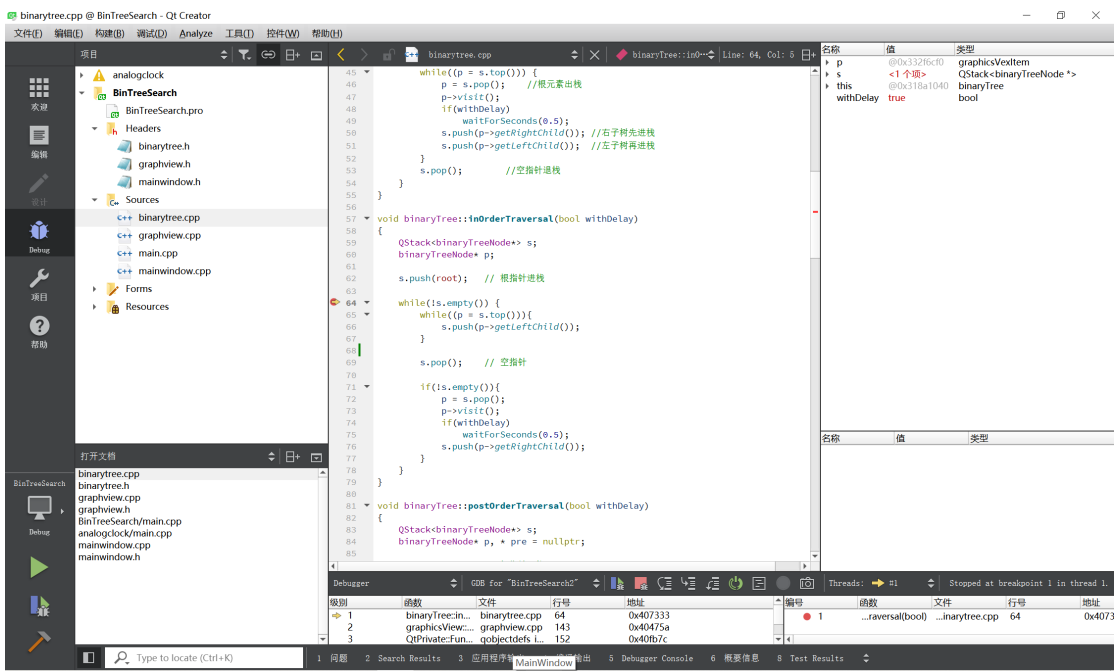


如上图所示，通过 qDebug 可以在程序运行期间在控制台打印所需要的调试信息，辅助

我进行问题的排查与定位，更快速的实现程序的调试与逻辑的改进。

2. 通过 Qt 的调试模式进行调试分析

Qt 提供了强大的调试工具，包括调试器和分析工具。在开发过程中，使用 Qt 的调试模式来单步执行代码、观察变量值和查看函数调用堆栈。这有助于深入了解程序的运行状态，以及在出现问题时快速定位错误所在。通过调试模式，可以逐步验证算法的正确性，并进行必要的修复和优化。



如上图所示，通过 Qt 的调试器，可以进行单步调试。在进行二叉树的遍历过程中，可以在逐步调试的过程中方便的定位到问题所在。我通过 Qt 的调试工具解决了许多包括空指针问题在内的内存问题导致的程序异常终止。

1.6.2 软件正确性、稳定性与容错能力

• 软件正确性：

通过测试和验证，确保了软件在各种情况下都能按照预期执行。针对二叉树的建立、遍历、线索化等功能，进行了一系列测试用例的测试，覆盖了各种可能的输入情况。这些测试用例有助于验证算法的正确性，确保软件能够正确处理不同的二叉树结构。该部分的具体测试将在后文中进行说明阐述。

• 软件稳定性：

在软件的开发过程中，我特别关注了内存管理和异常处理。通过了解与应用 Qt 对象树的概念，程序实现了内存资源的自动管理。Qt 对象树保证了对象的生命周期与父对象的生命周期一致，从而有效避免了内存泄漏和资源泄漏的问题。下面我举例子对 Qt 对象树在本课程设计中的应用进行说明：

1. graphicsVexItem 类的继承

graphicsVexItem 类继承了 QObject 类和 QGraphicsEllipseItem 类，同时也实现了 binaryTreeNode 类的接口。这意味着 graphicsVexItem 对象可以成为 Qt 对象树中的一部分，可以有一个父对象，也可以包含子对象。这种继承关系体现了 Qt 对象树的特性。在通过 QGraphicsScene 的 addItem 方法将 graphicsVexItem 实例添加到场景中时，也绑定了对应的对象树关系，在析构 QGraphicsScene 时，其中的所有 item 也会被递归地析构。

2. 对象的生命周期管理

```
void graphicsView::handleClearCanvas() {  
    // ...  
    // 清空图形视图中的所有对象  
    graphicsScene->clear();  
}
```

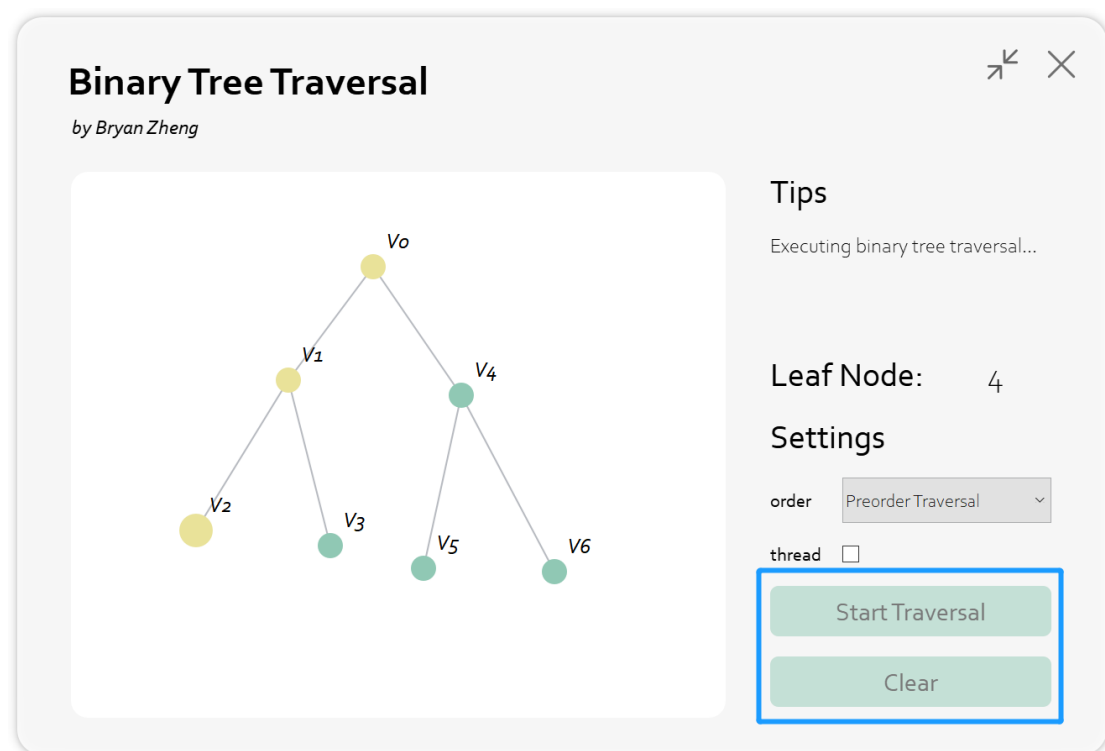
如前文所述，graphicsVexItem 对象的生命周期与图形视图 (graphicsScene) 相关联。例如，当你创建一个新的 graphicsVexItem 对象并添加到图形视图中时，图形视图成为该对象的父对象。当清空图形视图时，所有与之相关的 graphicsVexItem 对象也会被销毁，从而释放相关的内存资源。此外，在通过 removeItem 进行移除元素时，由于 Qt 不会自动析构对应的 item，我也特别注意对每个被移除的元素进行单独的空间释放：

```
inline void graphicsView::removeThread()  
{  
    graphicsThreadItem* thread;  
    while(!threads.empty()){  
        thread = threads.back();  
        threads.pop_back();  
        graphicsScene->removeItem(thread);  
        delete thread;    // 每个对象移除后额外进行释放  
    }  
}
```

- 软件容错能力:

由于采用了可视化的拖拽方式来建立二叉树,因此通过用户输入行为而导致的错误被大大减少。尽管如此,软件也具有一定的容错能力,能够处理用户可能出现的错误输入或异常操作。例如,每个根节点只能通过单击左键或者右键一次来拖拽出各一个的左右孩子,从而规避了用户建立的图形不是二叉树,乃至不是树型结构的隐患;当用户在没有选择根节点的情况下执行遍历操作时,软件并不会因为根节点为空陷入运行错误。这种容错设计有助于提高用户体验,减少用户因误操作而导致的问题。

在遍历二叉树时,若再拖动出新的结点,不仅会导致颜色不统一,也可能影响二叉树的正常线索化。因此二叉树遍历时,禁用了拖拽出新结点的功能。此外,如果在程序进行二叉树遍历的过程中清空画布,则在对应结点被析构时再次访问,就会导致运行错误。因此在遍历过程中,程序禁用了开始遍历和清空画布的按钮如下:



1.6.3 运行案例

下面以满二叉树为例,对建立二叉树、遍历二叉树、线索化二叉树的操作进行展示。

1. 二叉树的建立

Binary Tree Traversal

by Bryan Zheng

Tips

Click the canvas to create a root node.

Leaf Node: 0

Settings

order Preorder Traversal

thread ☐

Start Traversal

Clear

在空白画布上点击创建第一个结点：

Binary Tree Traversal

by Bryan Zheng

V_0



Tips

Click the node with left/right button to create a corresponding child node.

Leaf Node: 1

Settings

order Preorder Traversal

thread ☐

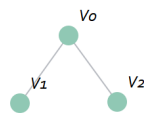
Start Traversal

Clear

分别通过左右键点击根节点创建左右子结点：

Binary Tree Traversal

by Bryan Zheng



Tips

Continue to click a node with left/right button to create a left/right child.

Leaf Node: 2

Settings

order Preorder Traversal

thread ☐

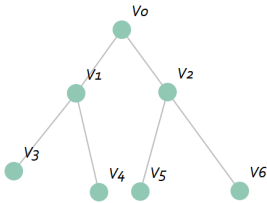
Start Traversal

Clear

继续创建完整颗树：

Binary Tree Traversal

by Bryan Zheng



Tips

Continue to click a node with left/right button to create a left/right child.

Leaf Node: 4

Settings

order: Preorder Traversal

thread: ☐

Start Traversal

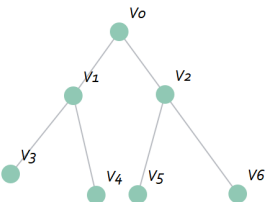
Clear

2. 叶子结点的个数统计

叶子结点个数信息统计正确：

Binary Tree Traversal

by Bryan Zheng



Tips

Continue to click a node with left/right button to create a left/right child.

Leaf Node: 4

Settings

order: Preorder Traversal

thread: ☐

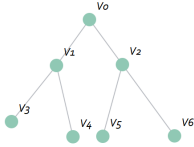
Start Traversal

Clear

3. 二叉树的前序遍历

Binary Tree Traversal

by Bryan Zheng



Tips

Continue to click a node with left/right button to create a left/right child.

Leaf Node: 4

Settings

order: Preorder Traversal

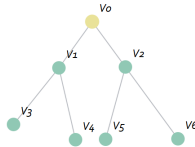
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Preorder Traversal

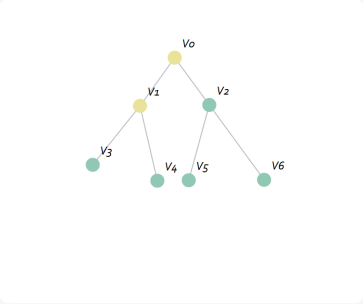
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Preorder Traversal

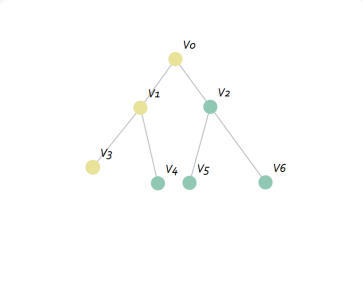
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Preorder Traversal

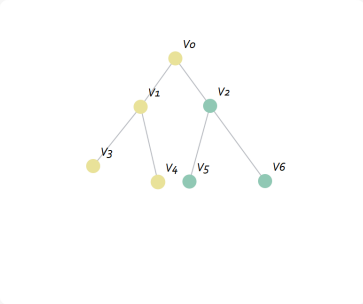
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Preorder Traversal

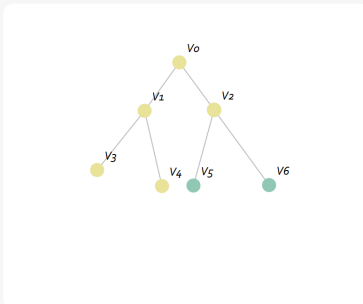
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Preorder Traversal

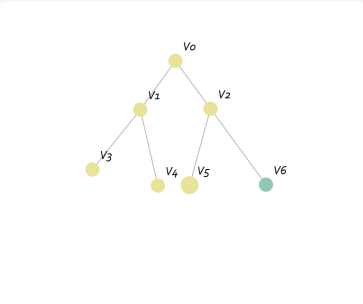
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Preorder Traversal

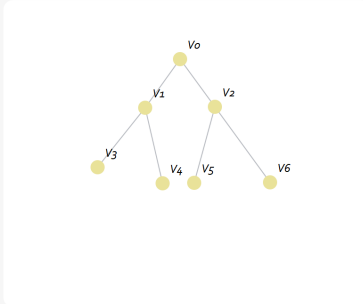
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

The traversal is done. Choose different mode to try again.

Leaf Node: 4

Settings

order: Preorder Traversal

thread: ☐

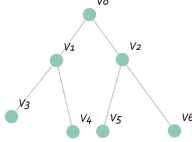
Start Traversal

Clear

4. 二叉树的中序遍历

Binary Tree Traversal

by Bryan Zheng



Tips
Continue to click a node with left/right button to create a left/right child.

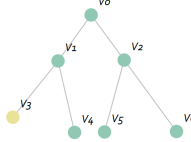
Leaf Node: 4

Settings
order: Preorder Traversal
thread: ☐

Start Traversal
Clear

Binary Tree Traversal

by Bryan Zheng



Tips
Executing binary tree traversal...

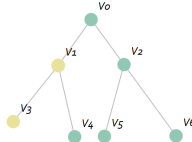
Leaf Node: 4

Settings
order: Inorder Traversal
thread: ☐

Start Traversal
Clear

Binary Tree Traversal

by Bryan Zheng



Tips
Executing binary tree traversal...

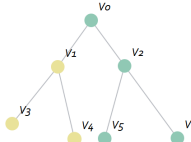
Leaf Node: 4

Settings
order: Inorder Traversal
thread: ☐

Start Traversal
Clear

Binary Tree Traversal

by Bryan Zheng



Tips
Executing binary tree traversal...

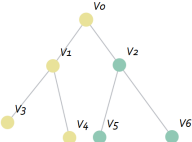
Leaf Node: 4

Settings
order: Inorder Traversal
thread: ☐

Start Traversal
Clear

Binary Tree Traversal

by Bryan Zheng



Tips
Executing binary tree traversal...

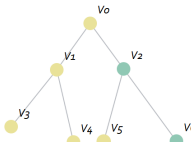
Leaf Node: 4

Settings
order: Inorder Traversal
thread: ☐

Start Traversal
Clear

Binary Tree Traversal

by Bryan Zheng



Tips
Executing binary tree traversal...

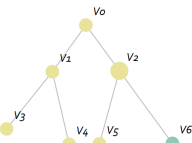
Leaf Node: 4

Settings
order: Inorder Traversal
thread: ☐

Start Traversal
Clear

Binary Tree Traversal

by Bryan Zheng



Tips
Executing binary tree traversal...

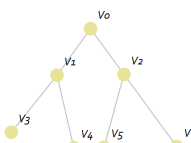
Leaf Node: 4

Settings
order: Inorder Traversal
thread: ☐

Start Traversal
Clear

Binary Tree Traversal

by Bryan Zheng



Tips
The traversal is done. Choose different mode to try again.

Leaf Node: 4

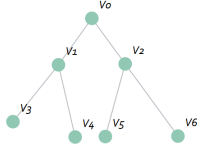
Settings
order: Inorder Traversal
thread: ☐

Start Traversal
Clear

5. 二叉树的后序遍历

Binary Tree Traversal

by Bryan Zheng



Tips

The traversal is done. Choose different mode to try again.

Leaf Node: 4

Settings

order: Postorder Traversal

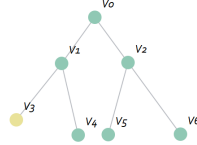
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Postorder Traversal

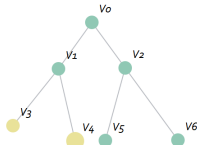
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Postorder Traversal

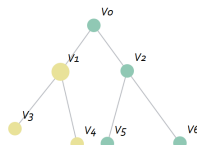
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Postorder Traversal

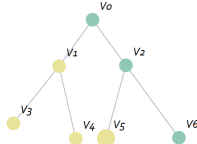
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Postorder Traversal

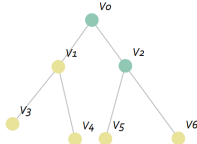
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Postorder Traversal

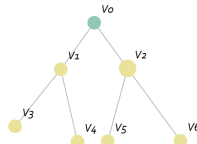
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 4

Settings

order: Postorder Traversal

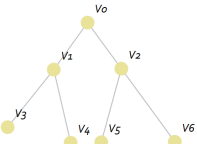
thread: ☐

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

The traversal is done. Choose different mode to try again.

Leaf Node: 4

Settings

order: Postorder Traversal

thread: ☐

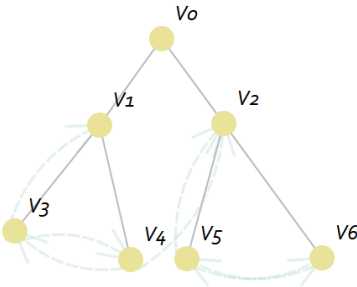
Start Traversal

Clear

6. 二叉树的前序线索化与遍历线索化树

Binary Tree Traversal

by Bryan Zheng



Tips

The traversal is done. Choose different mode to try again.

Leaf Node: 4

Settings

order: Preorder Traversal

thread: ☒

Create & Traverse

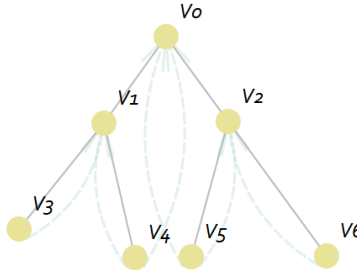
Clear

上图展示了前序线索化后的二叉树。在前序线索化后，程序自动执行线索化二叉树的遍历，因过程与前文类似，此处不再赘述。

7. 二叉树的中序线索化与遍历线索化树

Binary Tree Traversal

by Bryan Zheng



Tips

The traversal is done. Choose different mode to try again.

Leaf Node: 4

Settings

order: Inorder Traversal

thread: ☒

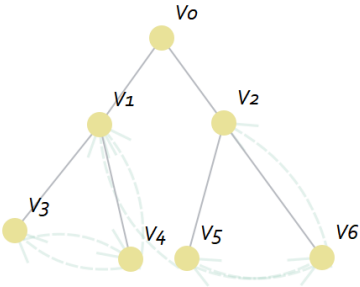
Create & Traverse

Clear

上图展示了中序线索化后的二叉树。在中序线索化后，程序自动执行线索化二叉树的遍历，因过程与前文类似，此处不再赘述。

Binary Tree Traversal

by Bryan Zheng



Tips

The traversal is done. Choose different mode to try again.

Leaf Node: 4

Settings

order Postorder Traversal

thread ☒

Create Tree

Clear

8. 二叉树的后序线索化

9. 清空画布

Binary Tree Traversal

by Bryan Zheng

Tips

Click the canvas to create a root node.

Leaf Node: 0

Settings

order Postorder Traversal

thread ☒

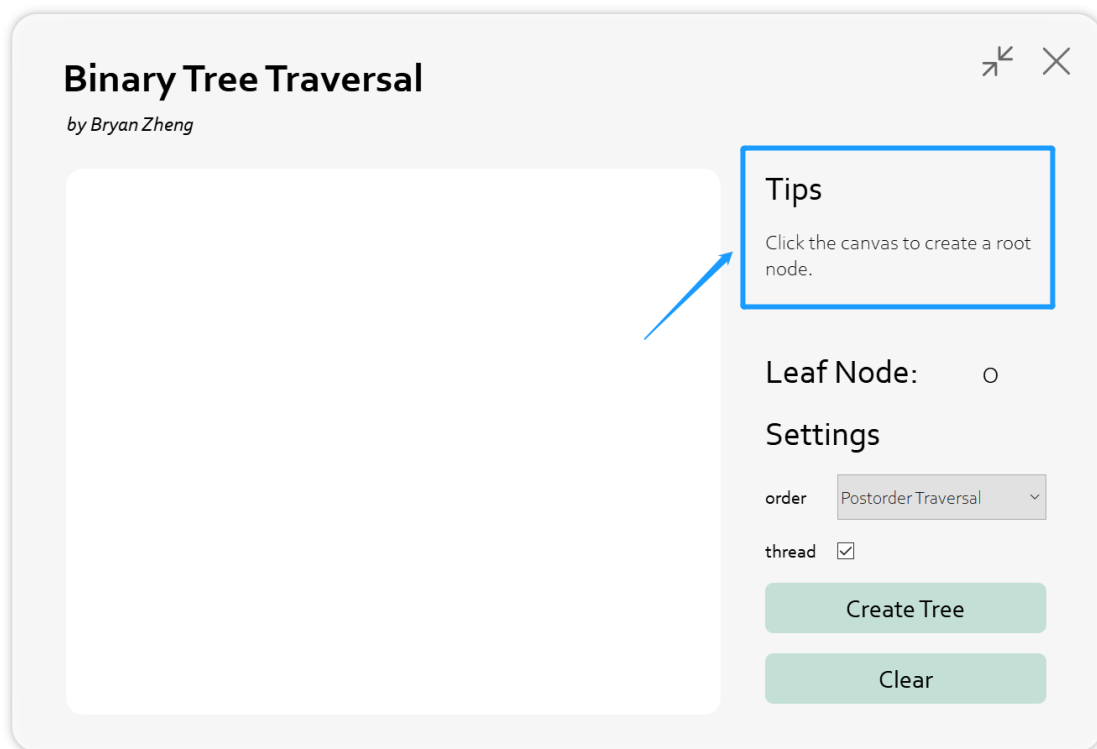
Create Tree

Clear

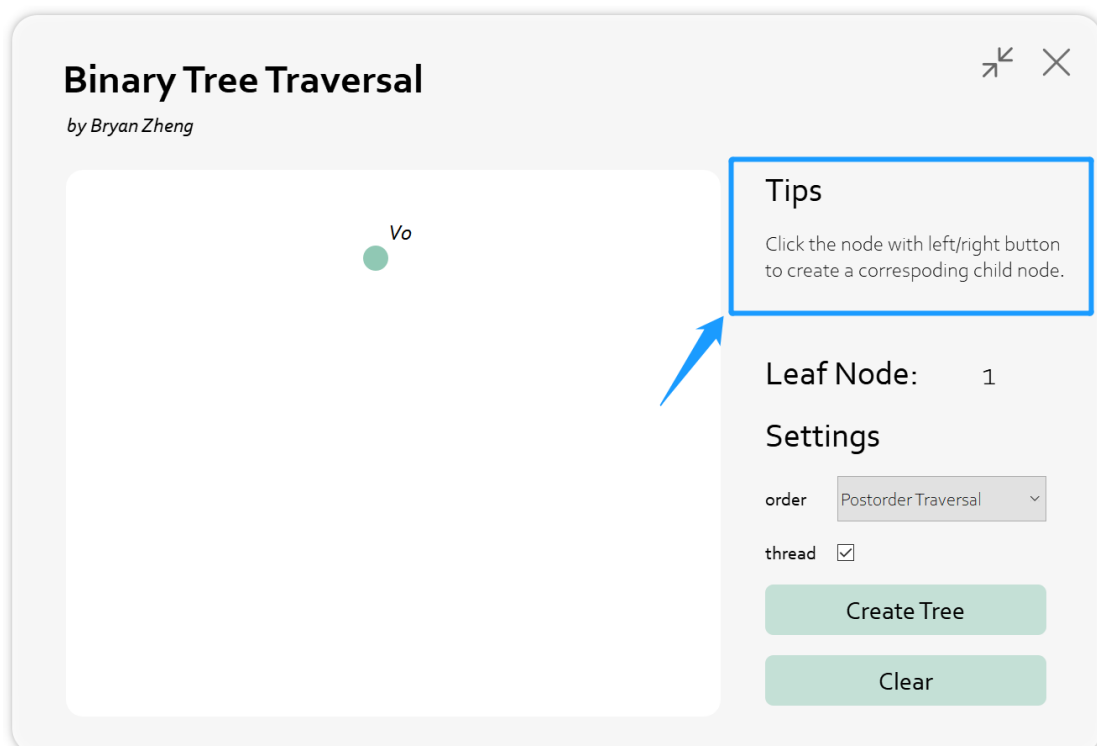
1.7 操作说明

1.7.1 二叉树的建立

初始画布为空时，右侧提示栏会给出点击画布创建根节点的提示：



点击画布任意位置，可以创建出根节点。此时提示栏给出创建子节点提示。



以鼠标左键点击根节点后，可以拖拽出连边，提示栏给出单击创建子结点提示：

Binary Tree Traversal

by Bryan Zheng



鼠标位置

Tips

Drag and click again to create a new node.

Leaf Node: 1

Settings

order Postorder Traversal

thread ☒

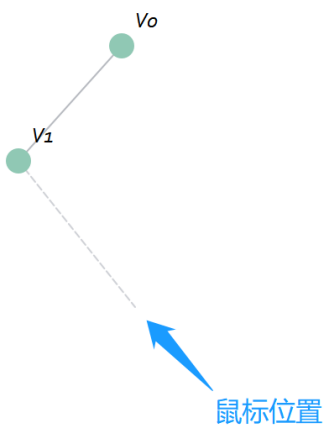
Create Tree

Clear

同理，以鼠标右键单击结点后，可以拖拽出右子结点：

Binary Tree Traversal

by Bryan Zheng



鼠标位置

Tips

Drag and click again to create a new node.

Leaf Node: 1

Settings

order Postorder Traversal

thread ☒

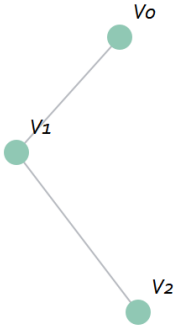
Create Tree

Clear

1.7.2 叶子结点的统计

Binary Tree Traversal

by Bryan Zheng



Tips

Continue to click a node with left/right button to create a left/right child.

Leaf Node: 1

Settings

order: Postorder Traversal

thread: ☒

Create Tree

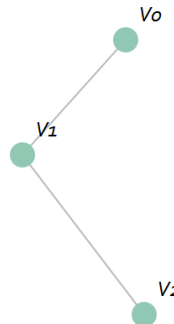
Clear

叶子结点的数量信息实时统计，显示在窗口右侧。

1.7.3 以不同模式进行二叉树遍历

Binary Tree Traversal

by Bryan Zheng



Tips

Continue to click a node with left/right button to create a left/right child.

Leaf Node: 1

Settings

order: Postorder Traversal

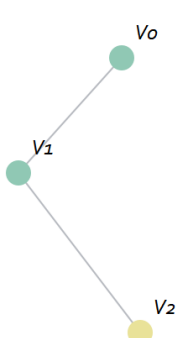
thread: ☒

Start Traversal

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Executing binary tree traversal...

Leaf Node: 1

Settings

order Postorder Traversal

thread ☐

Start Traversal

Clear

点击右侧下拉框可以选择遍历模式。

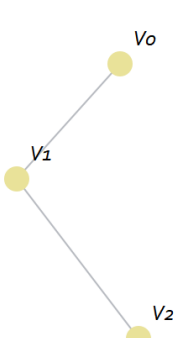
点击下方开始遍历按钮，以动画形式展示二叉树遍历。

1.7.4 进行二叉树线索化

选中“线索化”选项，切换到线索化模式。

Binary Tree Traversal

by Bryan Zheng



Tips

The traversal is done. Choose different mode to try again.

Leaf Node: 1

Settings

order Inorder Traversal

thread ☒

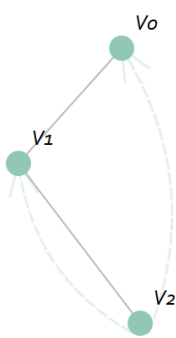
Create & Traverse

Clear

点击创建并遍历按钮，显示出线索后的二叉树：

Binary Tree Traversal

by Bryan Zheng



Tips

The threaded binary tree creation is completed.

Leaf Node: 1

Settings

order Inorder Traversal

thread ☒

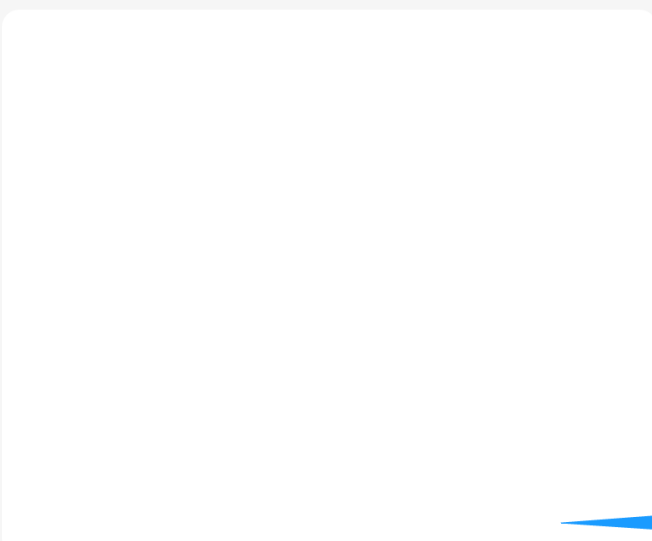
Create & Traverse

Clear

1.7.5 清空画布与重绘

Binary Tree Traversal

by Bryan Zheng



Tips

Click the canvas to create a root node.

Leaf Node: 0

Settings

order Inorder Traversal

thread ☒

Create & Traverse

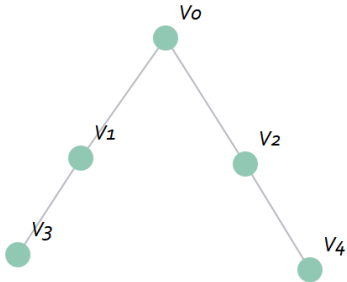
Clear

点击清空画布按钮后，左侧画布的二叉树被清空。

清空画布后，仍然可以正常建立新的二叉树并进行遍历和线索化：

Binary Tree Traversal

by Bryan Zheng



Tips

Continue to click a node with left/right button to create a left/right child.

Leaf Node: 2

Settings

order Inorder Traversal

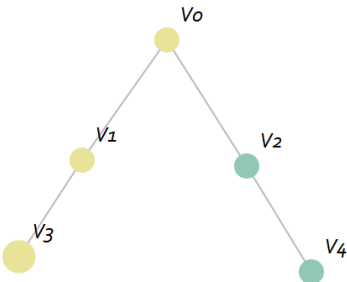
thread ☒

Create & Traverse

Clear

Binary Tree Traversal

by Bryan Zheng



Tips

Creating a threaded binary tree...

Leaf Node: 2

Settings

order Inorder Traversal

thread ☒

Create & Traverse

Clear

第二部分 综合应用设计说明

2.1 题目

上海的地铁交通网路已基本成型，建成的地铁线十多条，站点上百个，现需建立一个换乘指南打印系统，通过输入起点站和终点站，打印出地铁换乘指南，指南内容包括起点站、换乘站、终点站。

- （1）图形化显示地铁网络结构，能动态添加地铁线路和地铁站点。
- （2）根据输入起点站和终点站，显示地铁换乘指南。
- （3）通过图形界面显示乘车路径。

2.2 软件功能

2.2.1 地铁信息查看功能

• 功能描述

这个功能允许用户在图形界面上可视化查看上海地铁的线路，并能够通过鼠标拖动和滚轮来缩放和移动地铁网络地图。同时，用户还可以将鼠标悬浮在地铁站点上，以查看有关该站点的详细信息，如站点名称、位置等。

• 图形化界面说明

1. 用户启动程序后，会看到一个包含上海地铁线路的可视化地铁网络地图。
2. 地铁线路以不同颜色或样式表示，各站点沿线显示。
3. 用户可以通过鼠标拖动地图来移动视图，以便查看不同区域的线路。
4. 用户还可以使用鼠标滚轮来缩放地图，以查看线路的细节。
5. 当用户将鼠标悬浮在地铁站点上时，会弹出一个信息窗口，显示站点的相关信息，如站点名称、位置坐标、线路信息等。

• 实现方式

1. 创建一个 `graphicsView` 类，该类作为地铁网络地图的可视化界面，继承自

QGraphicsView 类。

2. 利用从配置文件中读取的地铁网络数据，在界面上绘制各地铁线路的路径和站点信息。

3. 实现鼠标事件来捕捉用户的拖动、滚轮操作以及悬浮操作，以实现地图的缩放、移动和站点信息的查看。

4. 通过悬浮操作，检测鼠标所在的站点，并在信息窗口中显示站点的详细信息。

2.2.2 地铁网络编辑功能

• 功能描述

用户可以通过图形界面动态添加地铁线路和站点，以维护地铁网络的数据。

• 图形化界面说明

1. 用户可以通过点击“添加站点”，在弹出的对话框中输入新站点的信息。确认无误后，可以点击画布的某个位置后，一个带有站点名称的图标会在该位置出现，成功新建地铁站点。

2. 用户可以通过点击“添加线路”，创建已有站点直接的地铁线路连接关系。

3. 用户可以增加新的地铁线路，指定线路名称与图例颜色。

• 实现方式

1. 创建一个 `graphicsView` 类，该类作为画布，继承自 `QGraphicsView` 类，用户可以通过在画布上点击鼠标来创建地铁站点。

2. 创建一个 `graphicsStationItem` 类，该类表示地铁站点，继承自 `QGraphicsEllipseItem` 类，用于绘制站点的图形表示。

3. 利用自定义的数据结构来保存用户创建的站点和地铁线路信息，并在内部建立相应的关系，以构建完整的地铁网络数据。

4. 使用鼠标事件捕捉用户的点击拖动操作，根据操作来创建地铁站点和连接地铁线路。

2.2.2 换乘指南生成功能

• 功能描述

根据用户输入的起点站和终点站，生成地铁换乘指南，指南内容包括起点站、换乘站、终点站。

• 图形化界面说明

1. 用户在界面上输入起点站和终点站。
2. 用户点击“生成指南”按钮后，程序会计算并显示地铁换乘指南，可以选择不同模式得到最佳换乘路径和相关信息。

• 实现方式

1. 用户输入的站点信息将与地铁网络数据进行匹配，以确定起点站和终点站在地铁网络中的位置。
2. 利用 Dijkstra 单源最短路径算法计算最佳的换乘路径^{[6][7][8][9]}，确定换乘站。
3. 构建换乘指南的文本信息，包括起点站、换乘站、终点站，并在界面上显示。

2.2.4 图形界面显示乘车路径

• 功能描述

在图形界面上显示用户的乘车路径，以便用户更直观地了解从起点站到终点站的路径。

• 图形化界面说明

1. 用户在生成地铁换乘指南后，可以在图形界面上看到完整的乘车路径，包括沿途的站点和线路。

2. 乘车路径上的站点和线路可能以高亮来表示，以提示用户体验。

• 实现方式

1. 利用地铁网络数据和生成的地铁换乘指南信息，绘制用户的乘车路径。
2. 使用 Qt 的绘图功能，将路径以图形方式展示在画布上，包括站点和线路的绘制。

2.3 设计思想

在设计本次的上海地铁换乘软件中，我采用了面向对象的设计思想，将地铁地图的各个组成部分抽象为不同的类，以便进行高效的数据管理和可视化。同时，我也尝试着遵循着 MVC（Model-View-Controller）设计模式，以更好地组织和分离系统的不同层次。下面是本次设计思路 and 核心要点：

2.3.1 类的设计与抽象^[10]

我定义了以下核心类来组织和管理地铁地图数据：

- **metroMapData 类（Model）**：表示整个地铁地图，包含了地铁线路、站点、边缘等信息，提供了一系列方法来读取、写入、搜索路径等。这个类充当了系统的核心数据模型，负责维护地铁地图的逻辑结构。

- **metroStationData 类（Model）**：表示单个地铁站点，包括站点的基本信息（名称、位置、类型等）以及连接的边缘列表。每个 **metroStationData** 实例代表一个站点，将站点的信息封装在内部。

- **metroEdgeData 类（Model）**：表示站点之间的连接，包括起始站点、结束站点、线路名称、颜色等信息。这个类用于表示站点之间的关系。

- **metroLineData 类（Model）**：表示地铁线路，包括线路的名称和颜色。每个 **metroLineData** 实例代表一条地铁线路。

- **metroGraphicsView 类（View）**：图形界面类，这是 View 部分的顶层类，负责管理和渲染地铁地图的可视化元素，存储下述站点、连边的可视化信息。

- **metroGraphicsStationItem 类（View）**：可视化站点类，用于在图形界面中表示地铁站点。这个类接收 **metroStationData** 数据并在画布上绘制站点。

- **metroGraphicsEdgeItem 类（View）**：可视化连边类，用于在图形界面中表示地铁站点之间的连接。这个类接收 **metroEdgeData** 数据并在画布上绘制站点连边。

2.3.2 数据结构的选择

在设计地铁地图数据结构时，我采用了与图的邻接表存储方式类似的设计，用于有效地管理站点和它们之间的边缘关系。邻接表是一种常见的图数据结构，它将每个顶点的相邻的

边列表存储在一个链表或向量中。由于地铁站名称是唯一的（4 号线与 6 号线浦电路的名称重合通过后缀以区分，此处不纳入考虑），可以使用 map 结构进行映射。

我使用了 QMap 数据结构来存储地铁站点的信息。在这里，每个站点的唯一名称被用作键，而与该站点相关联的数据被存储为值。这类似于邻接表中的顶点部分，其中每个顶点有一个唯一的标识符，在本次设计中我直接用站点名称作为键值，而无需通过下标查找。

为了表示站点之间的连接关系，我使用了 QVector 来存储每个站点相邻的边缘的信息。这部分类似于邻接表中的链表部分，QVector 存储与站点相邻的连边的信息。每个站点对象中都包含了一个 QVector，其中存储了与该站点相邻的边缘。这种方式使得在查找某个站点相邻的边缘时能够高效地完成操作。

此外，在使用 Dijkstra 算法进行单源最短路径的求算时^{[11][12]}，本程序还使用了优先队列（priority queue）这一数据结构，从而使得当前距离起始站点最近的站点能优先弹出。

2.3.3 算法的基本流程

1. 读取配置文件：在 metroMapData 类的 readConfig 方法中，首先打开配置文件，然后使用 QJsonDocument 和 QJsonObject 解析文件中的 JSON 数据，以获取地铁线路、站点和边缘的信息。然后，我们将这些信息填充到相应的数据结构中。

2. 查找路径：metroMapData 类提供了多个查找路径的方法，包括根据站点数量、距离和换乘次数等不同的策略。其中，最短路径算法使用了 Dijkstra 算法的变体，通过优先队列实现。算法首先选择起始站点，然后遍历与其相邻的站点，计算到达每个相邻站点的距离，并使用优先队列来选择下一个探索的站点，直到找到目标站点或遍历完所有可能的路径。

3. 写入配置文件：在 metroMapData 类的 writeConfig 方法中，将地铁地图数据转换为 JSON 格式，然后将其写入配置文件。这包括线路、站点、边缘的信息。

2.4 逻辑结构与物理结构

2.4.1 逻辑结构

地铁线路的网状关系可以容易地抽象为图状结构。在本次程序中，采用了图状结构来表示地铁地图的逻辑关系。这个逻辑结构基于图的概念，其中地铁站点被视为图中的顶点，而站点之间的连接则被视为图中的边。这种图状结构有以下主要特点：

- **顶点：**地铁站点是图的顶点，每个站点代表一个地铁站，具有唯一的名称。这些站点之间通过边相互连接，形成了地铁网络。

- **边：** 地铁站点之间的连接关系由图的边来表示。每条边代表两个相邻的站点之间的地铁线路，包括线路名称和颜色信息。边是逻辑上的连接，用于表示站点之间的可达性。

- **图：** 整个地铁地图被抽象为一个图，其中包含了所有站点和站点之间的连接关系。图的结构用于表示站点之间的拓扑关系，以及用于路径搜索和可视化。

2.4.2 物理结构

在物理层面，本次程序采用了一种类似于邻接表的数据结构，用于有效地存储和管理地铁地图数据。本次程序使用了两种数据结构：`QMap` 和 `QVector`，修改后的结构并非链表这样的链式存储结构，具体修改如下：

- **Qmap 通过站名映射存储站点信息：**我使用了 QMap 数据结构来存储地铁站点的信息。在这里，每个站点的唯一名称被用作键，而与该站点相关联的数据被存储为值。这类似于邻接表中的顶点部分，其中每个顶点有一个唯一的标识符，在本次设计中我直接用站点名称作为键值，而无需通过下标查找。

- **Qvector 记录每个站点的连边：**为了表示站点之间的连接关系，我使用了 QVector 来存储每个站点相邻的边缘的信息。这部分类似于邻接表中的链表部分，QVector 存储与站点相邻的连边的信息。每个站点对象中都包含了一个 QVector，其中存储了与该站点相邻的边缘。这种方式使得在查找某个站点相邻的边缘时能够高效地完成操作。

其中涉及到的类定义具体如下：

```
// 存储地铁网络结构的类
class metroMapData
{
private:
    // 地铁线路信息
    QMap<QString, metroLineData *> metroLines;
    QVector<metroLineData *> metroLinesVec;    // 按线路顺序
    // 地铁站点信息
    QMap<QString, metroStationData *> metroStations;

    // 可视化地图的所有站点
```

```

    QVector<metroGraphicsStationItem *> metroGraphicStations;
    // 可视化地图的所有边
    QVector<metroGraphicsEdgeItem *> metroGraphicEdges;

    // 暂存的寻路路径
    QQueue<metroEdgeData *> path;

    // 配置信息
    const QString configPath = QString("./metroInfo.json");

    static int getDistance(QPointF start, QPointF end);

public:
    metroMapData();
    ~metroMapData();

    // 读写配置文件(成功 true, 失败 false)
    bool readConfig();
    bool writeConfig();

    // 获取 Map 基本信息
    QVector<metroLineData *> getMetroLines() const;
    QVector<metroGraphicsStationItem *> getMetroGraphicStations() const;
    QVector<metroGraphicsEdgeItem *> getMetroGrphicEdges() const;
    quint16 getStationNum();
    const QColor& getLineColor(QString name);
    QStringList getLinesName();
    bool isStationExisted(QString name);
    bool isLineExisted(QString name);

    // 地铁换乘寻路
    const QQueue<metroEdgeData *>& findPath(QString start, QString end,
    int (* getDistance)(const QQueue<metroEdgeData *>&, metroEdgeData *));
    static int getDistanceByCount(const QQueue<metroEdgeData *> & queue,
    metroEdgeData * edge);
    static int getDistanceByDistance(const QQueue<metroEdgeData *> &
    queue, metroEdgeData * edge);
    static int getDistanceByTransfer(const QQueue<metroEdgeData *> &
    queue, metroEdgeData * edge);

    // 用户增加新站点、线路
    metroGraphicsStationItem* addNewStation(metroStationData*
    newStation);

```

```
    metroGraphicsEdgeItem * addNewEdges(QString name1, QString name2,
    QString line);
    void addNewLine(QString name, QColor color);
};
```

```
// 存储地铁站信息的类
class metroStationData
{
    friend class metroMapData;
    friend class metroEdgeData;

    // 编号（唯一）与站点名称
    qint16 id;
    QString name;

    // 实际位置（经纬度）与地图位置
    QPointF realPosition;
    QPoint graphPosition;

    // 站点形态与标签位置
    STATION_TYPE type;
    NAMETAG_POSITION nametagPosition;

    // 与该站点相连的边
    QVector<metroEdgeData *> edges;

    // 对应的可视化站点
    metroGraphicsStationItem* graphicItem = nullptr;

public:
    metroStationData(qint16 _id, QString _name, QPoint _graphPos, QPointF
    _realPos, STATION_TYPE _type, NAMETAG_POSITION _tagPos);
    metroStationData(QString _name, QPointF _realPos, STATION_TYPE _type,
    NAMETAG_POSITION _tagPos);
    metroGraphicsStationItem* getGraphicItem() const;

    // 设置基础信息
    void setId(qint16 id);
    void setGraphPosition(const QPoint& position);
```

```
// 获取基础信息
const QString& getName() const;
const QPoint& getGraphPosition() const;
};
```

```
// 存储地铁边结构的类
class metroEdgeData
{
    friend metroMapData;

    // 起止信息
    QString startName, endName;
    metroStationData* start, * end;

    // 线路信息
    QString line;
    QColor lineColor;

    // 经过的点
    QVector<QPoint> via;

    // 对应的可视化边
    metroGraphicsEdgeItem* graphicItem;

public:
    metroEdgeData(QString _startName, QString _endName, QString _line,
        QColor _color, QVector<QPoint> _via);
    void fillInformation(const QMap<QString, metroStationData*>&
        metroStations); // 填充结点信息

    // 获取基本信息
    metroGraphicsEdgeItem* getGraphicItem() const;
    metroStationData* getStart() const;
    metroStationData* getEnd() const;
    const QString& getLine() const;
};
```

2.5 开发平台

计算机型号：联想小新 Pro 14ACH 2021

计算机内存：16.00GB

CPU：AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz

操作系统：Windows 10 家庭版

开发语言：C++（C++11 标准以上）

开发框架：QT

集成开发环境：Qt Version 5.12.12

编译器：MinGW 7.3.0 64-bit

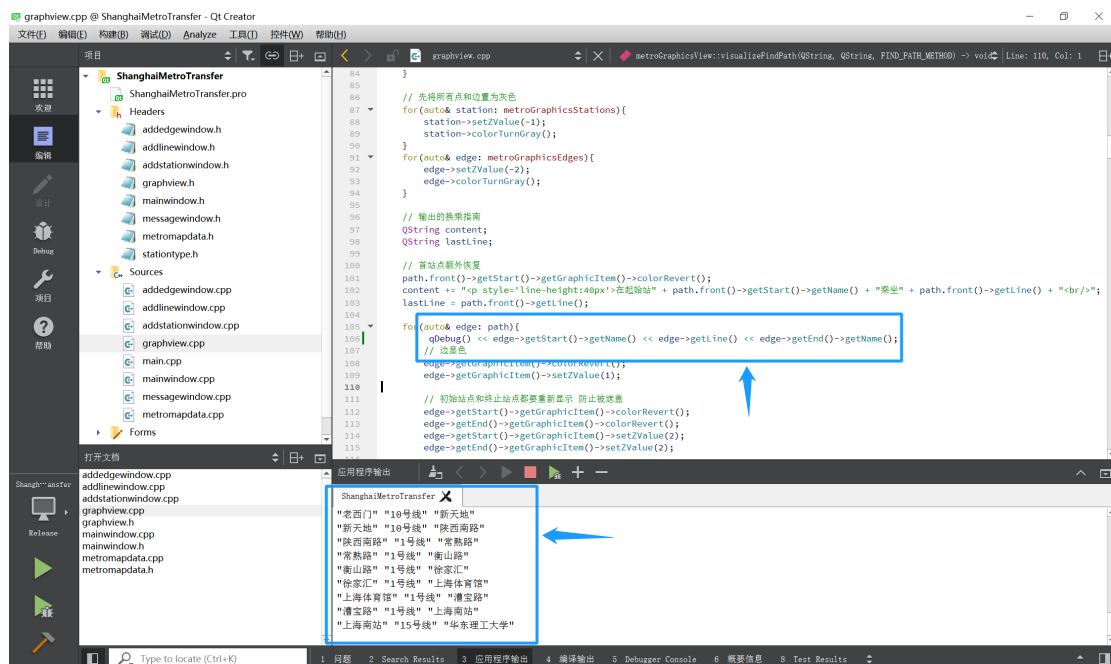
运行环境：通过 windeployqt.exe 及 Enigma Virtual Box 进行整合压缩为可执行文件，可在普通 windows 机型下运行。

2.6 系统的运行结果分析说明

2.6.1 调试与开发过程

1. 通过 qDebug 进行输出

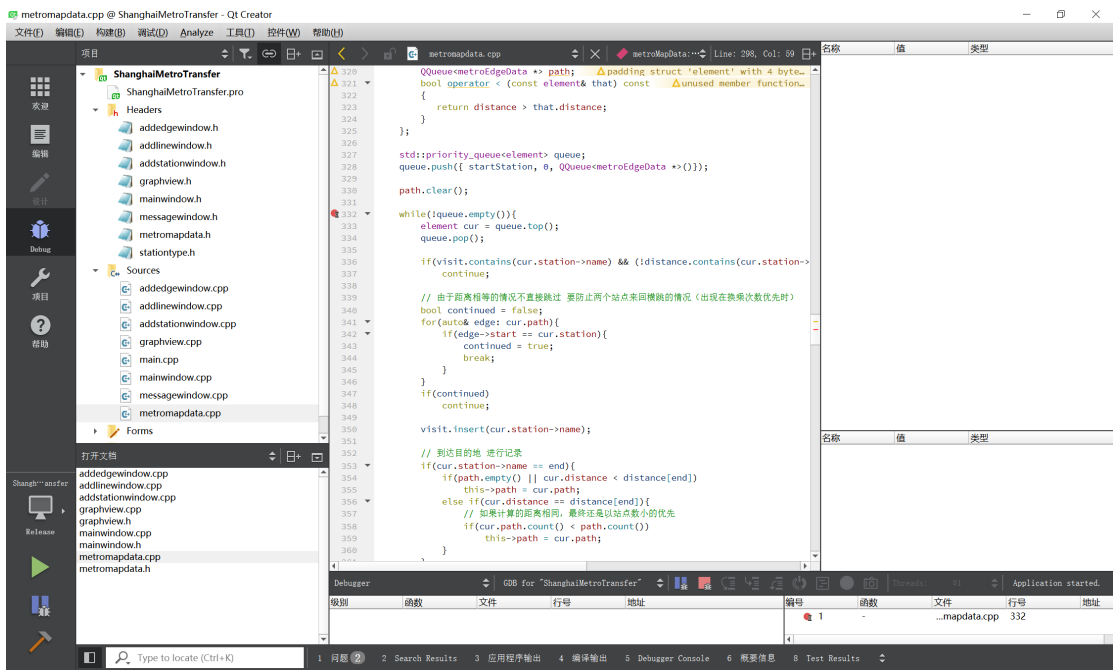
如下图所示，通过 qDebug 可以在程序运行期间在控制台打印所需要的调试信息，辅助我进行问题的排查与定位，更快速的实现程序的调试与逻辑的改进。



在开发过程中，我使用了 Qt 框架提供的 QDebug 输出来实时输出调试信息。这些输出信息包括了程序的执行流程、变量的值、以及函数的调用。通过 QDebug 输出，可以快速定位和解决潜在的问题，如内存泄漏、逻辑错误等。这些输出信息有助于我在开发过程中跟踪代码的执行路径，并确认算法的正确性。

2. 通过 Qt 的调试模式进行调试分析

Qt 提供了强大的调试工具，包括调试器和分析工具。在开发过程中，使用 Qt 的调试模式来单步执行代码、观察变量值和查看函数调用堆栈。这有助于深入了解程序的运行状态，以及在出现问题时快速定位错误所在。通过调试模式，可以逐步验证算法的正确性，并进行



必要的修复和优化。

如上图所示，通过 Qt 的调试器，可以进行单步调试。在进行换乘路径的 Dijkstra 算法搜索过程中，可以在逐步调试的过程中方便的定位到问题所在。我通过 Qt 的调试工具解决了许多包括空指针问题在内的内存问题导致的程序异常终止。

2.6.2 软件正确性、稳定性与容错能力

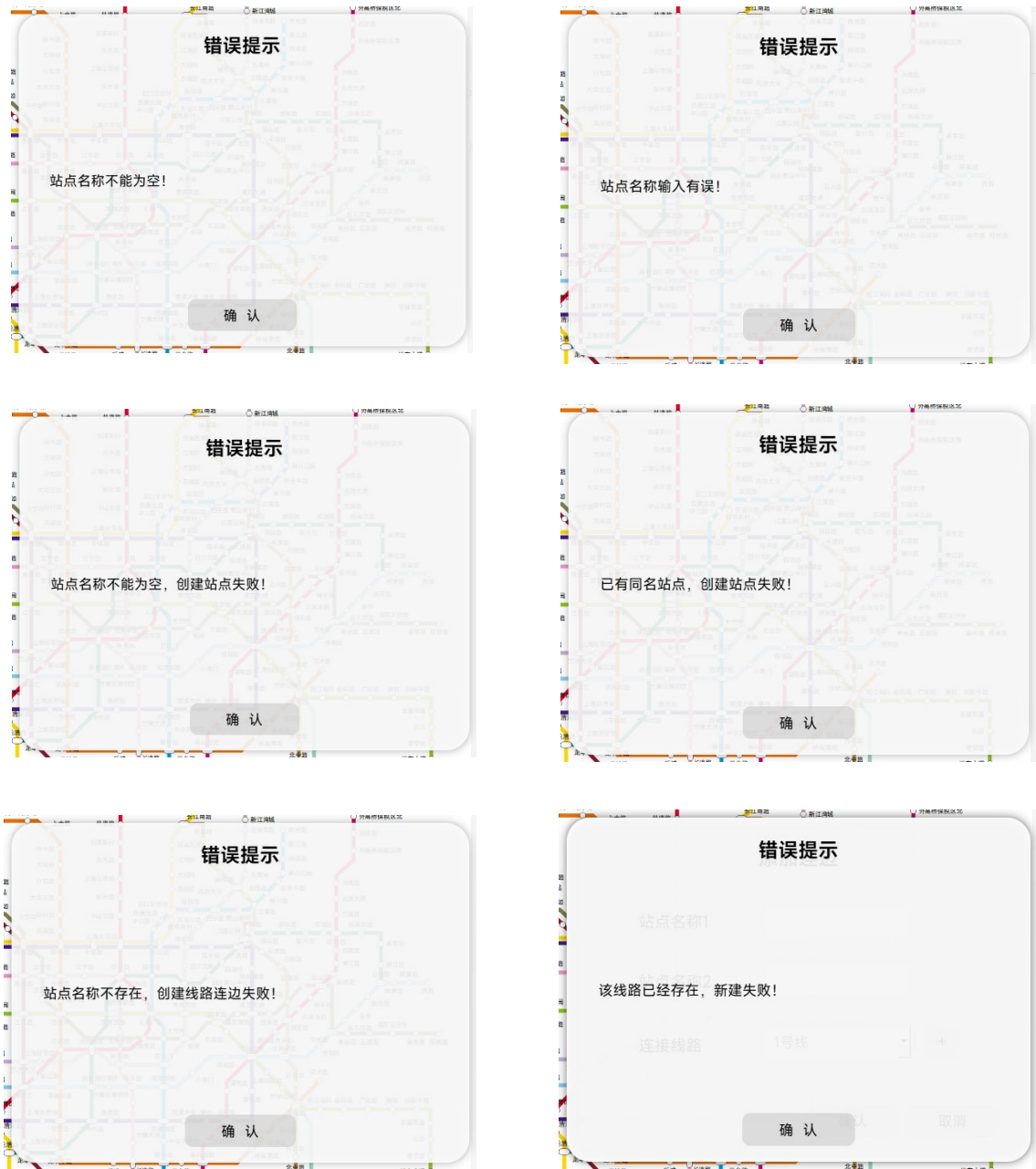
• 软件正确性：

通过测试和验证，确保了软件在各种情况下都能按照预期执行。我针对三种模式的换乘

路径选择，进行了一系列测试用例的测试，覆盖了各种可能的输入情况。这些测试用例有助于验证算法的正确性，确保软件能够正确处理不同的站点之间的寻路。该部分的具体测试将在后文中进行说明阐述。

• 软件稳定性与容错能力

本程序能够对多种错误情况进行错误处理,涵盖范围较广,能够保证软件运行的稳定性,具有较强的容错能力。其中,包含文件读取错误处理,即在读取地铁地图配置文件时,我使用了 Qt 提供的 JSON 解析工具。如果解析失败或文件损坏,程序会捕获异常并向用户显示有关错误的信息,同时不会导致崩溃。当用户在图形界面中输入站点或线路信息时,程序也会进行验证以确保输入的数据格式正确。如果用户提供了不合法的数据,程序会提供相应的错误提示,而不是尝试处理无效数据。错误提示包含如下图:



2.6.3 运行案例

1. 换乘寻路

- “昌吉东路” → “同济大学”

i. 站点数优先



ii. 距离优先

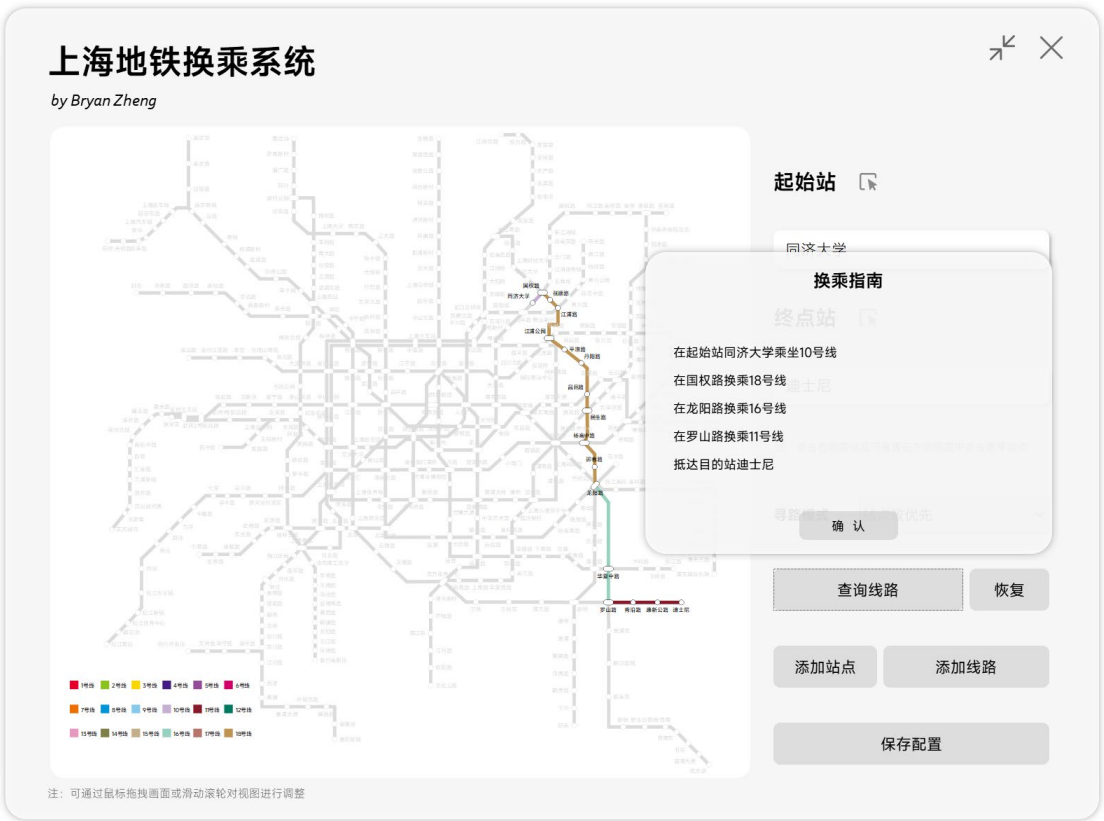


iii. 换乘次数优先



• “同济大学” → “迪士尼”

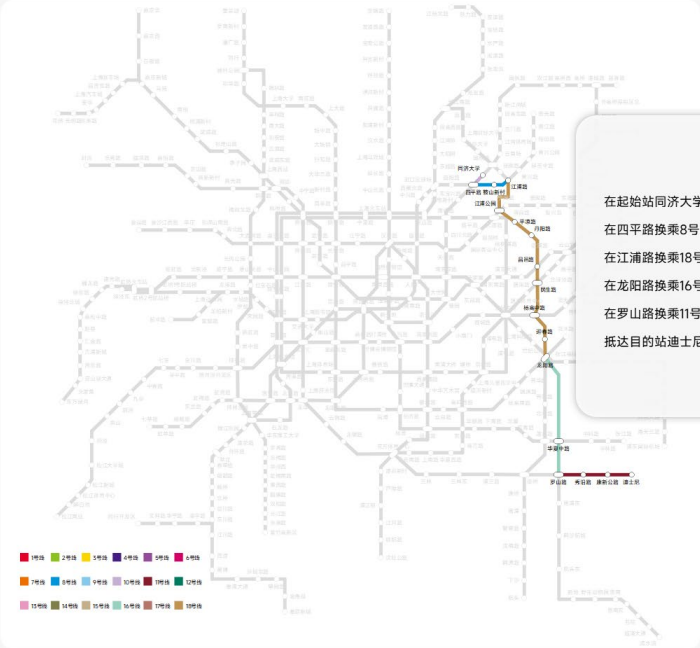
i. 站点数优先



ii. 距离优先

上海地铁换乘系统

by Bryan Zheng



注: 可通过鼠标拖拽画面或滑动滚轮对视图进行调整

起始站

换乘指南

在起始站同济大学乘坐10号线
在四平路换乘8号线
在江浦路换乘18号线
在龙阳路换乘16号线
在罗山路换乘11号线
抵达目的站迪士尼

确认

查询线路

恢复

添加站点

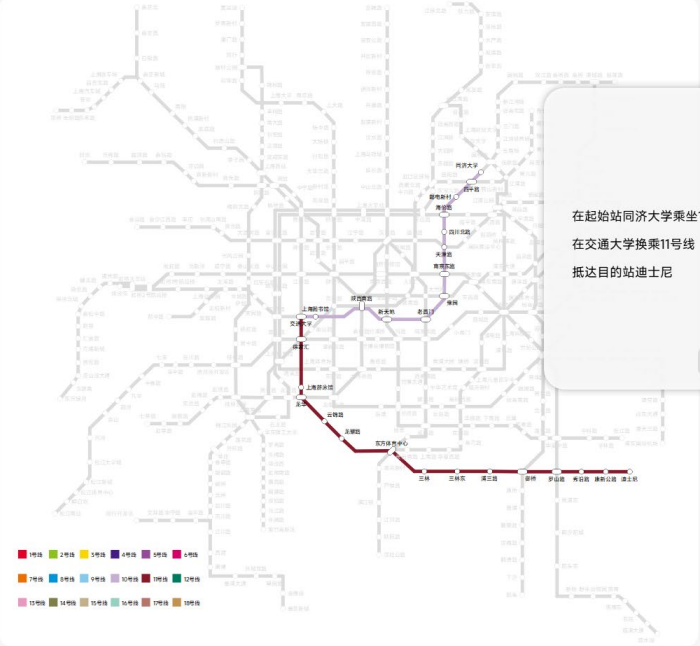
添加线路

保存配置

iii. 换乘次数优先

上海地铁换乘系统

by Bryan Zheng



注: 可通过鼠标拖拽画面或滑动滚轮对视图进行调整

起始站

换乘指南

在起始站同济大学乘坐10号线
在交通大学换乘11号线
抵达目的站迪士尼

确认

查询线路

恢复

添加站点

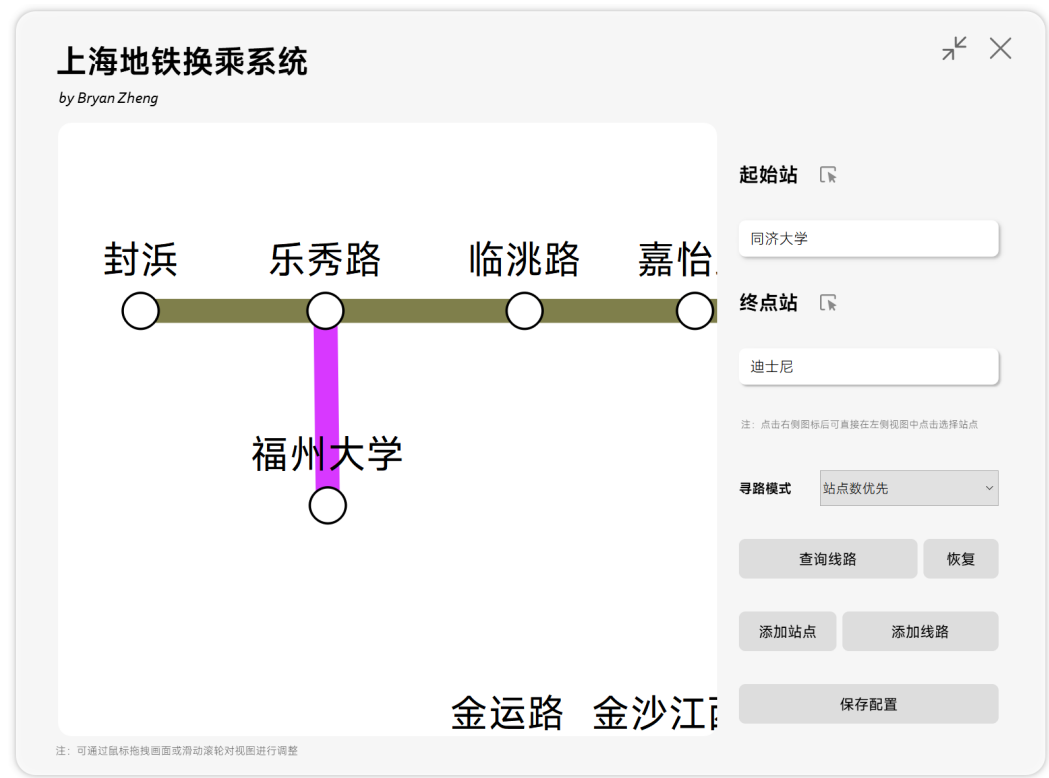
添加线路

保存配置

2. 添加站点



3. 添加线路



4. 新线路寻路

上海地铁换乘系统

by Bryan Zheng

起始站

换乘指南

终点站

在起始站同济大学乘坐10号线

在海伦路换乘4号线

在曹杨路换乘14号线

在乐秀路换乘福大路

抵达目的站福州大学

寻路模式 确认

查询线路 恢复

添加站点 添加线路

保存配置

注：可通过鼠标拖拽画面或滑动滚轮对视图进行调整

2.7 操作说明

上海地铁换乘系统

by Bryan Zheng

起始站

终点站

注：点击右侧图标后可直接在左侧视图中点击选择站点

寻路模式

查询线路 恢复

添加站点 添加线路

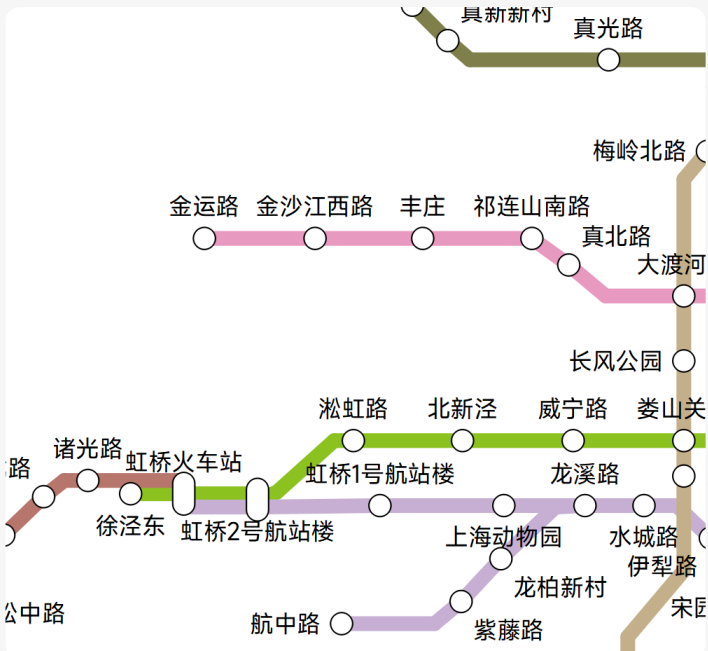
保存配置

注：可通过鼠标拖拽画面或滑动滚轮对视图进行调整

在左侧视图中可以拖拽鼠标、滑动滚轮进行视图的放缩：

上海地铁换乘系统

by Bryan Zheng



注：可通过鼠标拖拽画面或滑动滚轮对视图进行调整

起始站

终点站

注：点击右侧图标后可直接在左侧视图中点击选择站点

寻路模式

站点数优先

查询线路

恢复

添加站点

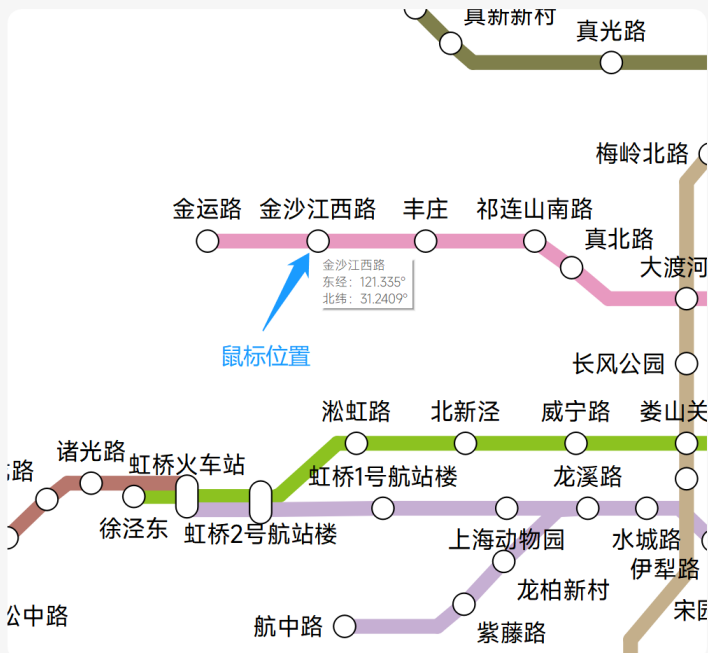
添加线路

保存配置

鼠标悬浮至站点上可以查看详细信息：

上海地铁换乘系统

by Bryan Zheng



注：可通过鼠标拖拽画面或滑动滚轮对视图进行调整

起始站

终点站

注：点击右侧图标后可直接在左侧视图中点击选择站点

寻路模式

站点数优先

查询线路

恢复

添加站点

添加线路

保存配置

可以在起始站、终点站输入框中输入站点名称，也可以点击右侧选择图标  后，再在左侧视图上点击选取站点：

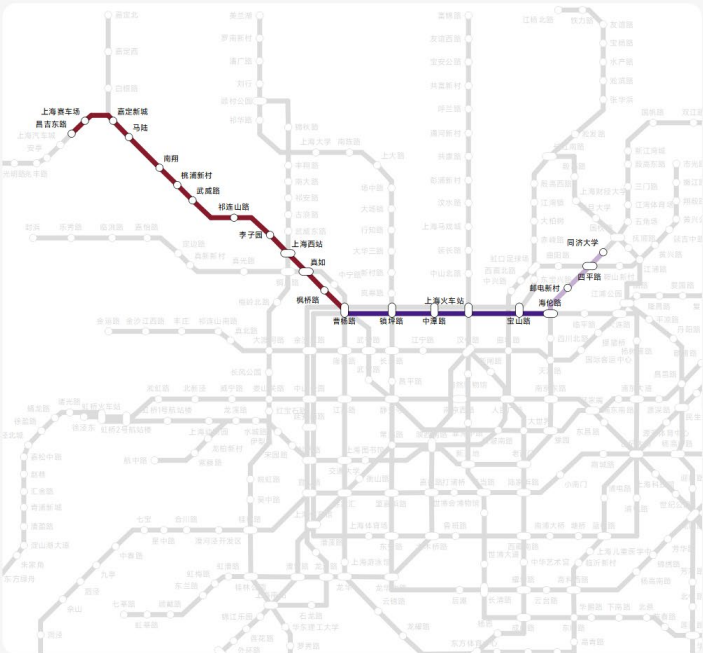


在下方寻路模式下拉框中可以选择三种不同的换乘寻路方式。点击查询线路后，会弹出换成指南，也会在视图上高亮显示换乘线路。



上海地铁换乘系统

by Bryan Zheng



注：可通过鼠标拖拽画面或滑动滚轮对视图进行调整

起始站

同济大学

终点站

昌吉东路

注：点击右侧图标后可直接在左侧视图中点击选择站点

寻路模式

站点数优先

查询线路

恢复

添加站点

添加线路

保存配置

点击“恢复”按钮，恢复地图上所有线路的高亮显示。

上海地铁换乘系统

by Bryan Zheng



注：可通过鼠标拖拽画面或滑动滚轮对视图进行调整

起始站

同济大学

终点站

昌吉东路

注：点击右侧图标后可直接在左侧视图中点击选择站点

寻路模式

站点数优先

查询线路

恢复

添加站点

添加线路

保存配置

点击“添加站点”按钮，弹出添加站点窗口。



点击确认后，程序给出提示信息：



上海地铁换乘系统

by Bryan Zheng

提示信息

站点记录成功, 请在线路图上挑选位置点击以放置站点!

确认

起始站

同济大学

终点站

昌吉东路

注: 点击右侧图标后可直接在左侧视图中点击选择站点

寻路模式

站点数优先

查询线路

恢复

添加站点

添加线路

保存配置

点击确认后, 程序给出提示信息:

在画布上适当位置单击, 就可以放置站点:

上海地铁换乘系统

by Bryan Zheng

美兰湖

罗南新村

潘广路

刘行

顾村公园

祁华路

锦秋路

上海大学 南陈路

丰翔路

南大路

福州大学

起始站

同济大学

终点站

昌吉东路

注: 点击右侧图标后可直接在左侧视图中点击选择站点

寻路模式

站点数优先

查询线路

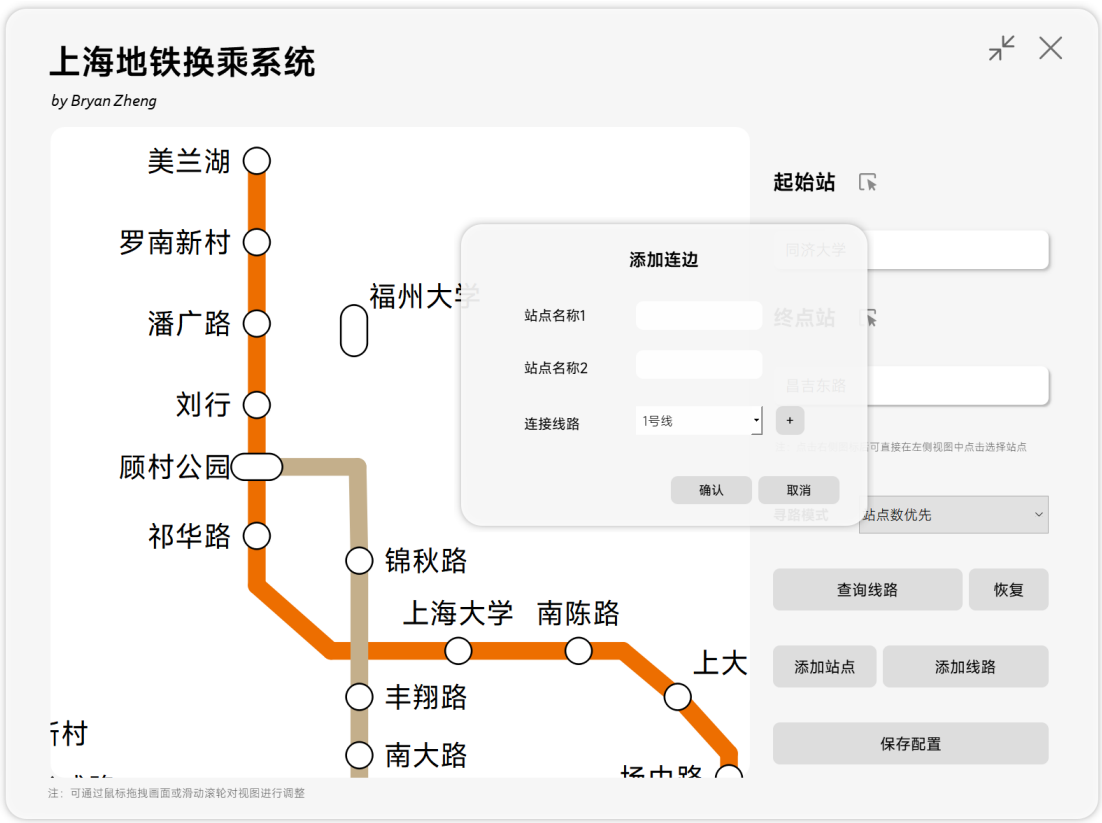
恢复

添加站点

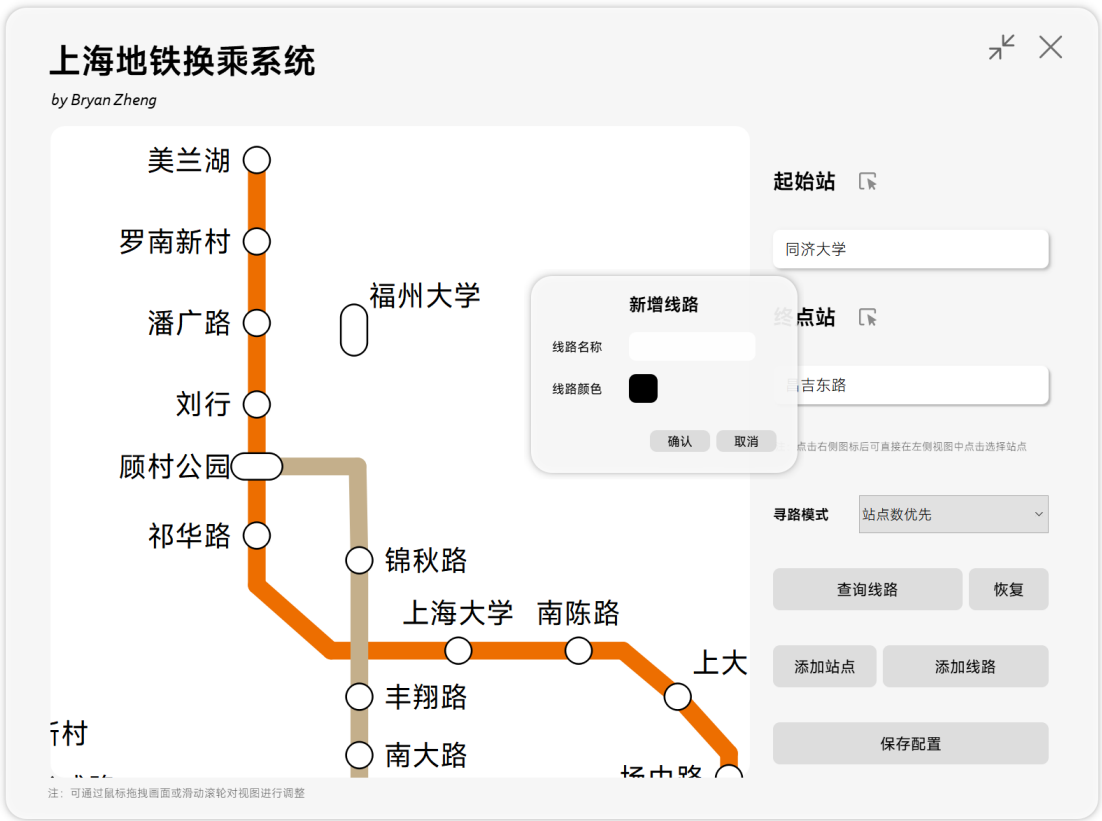
添加线路

保存配置

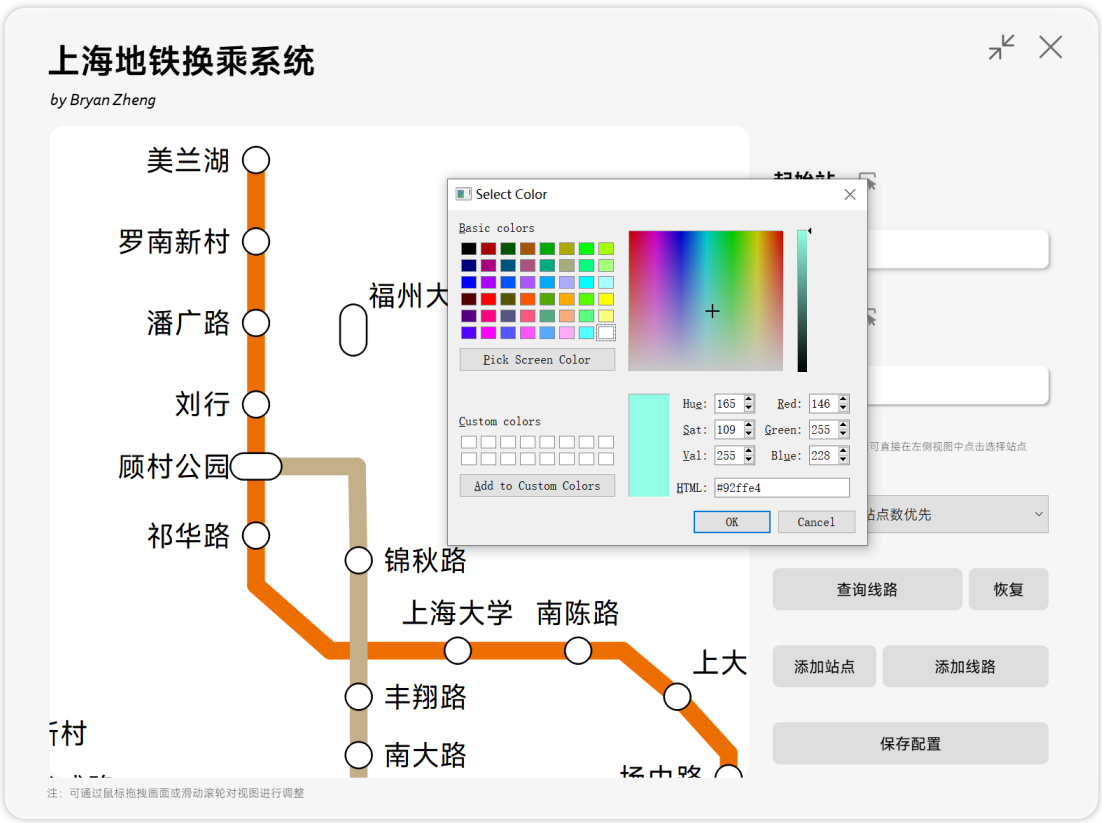
点击“添加线路”按钮，弹出对话框：



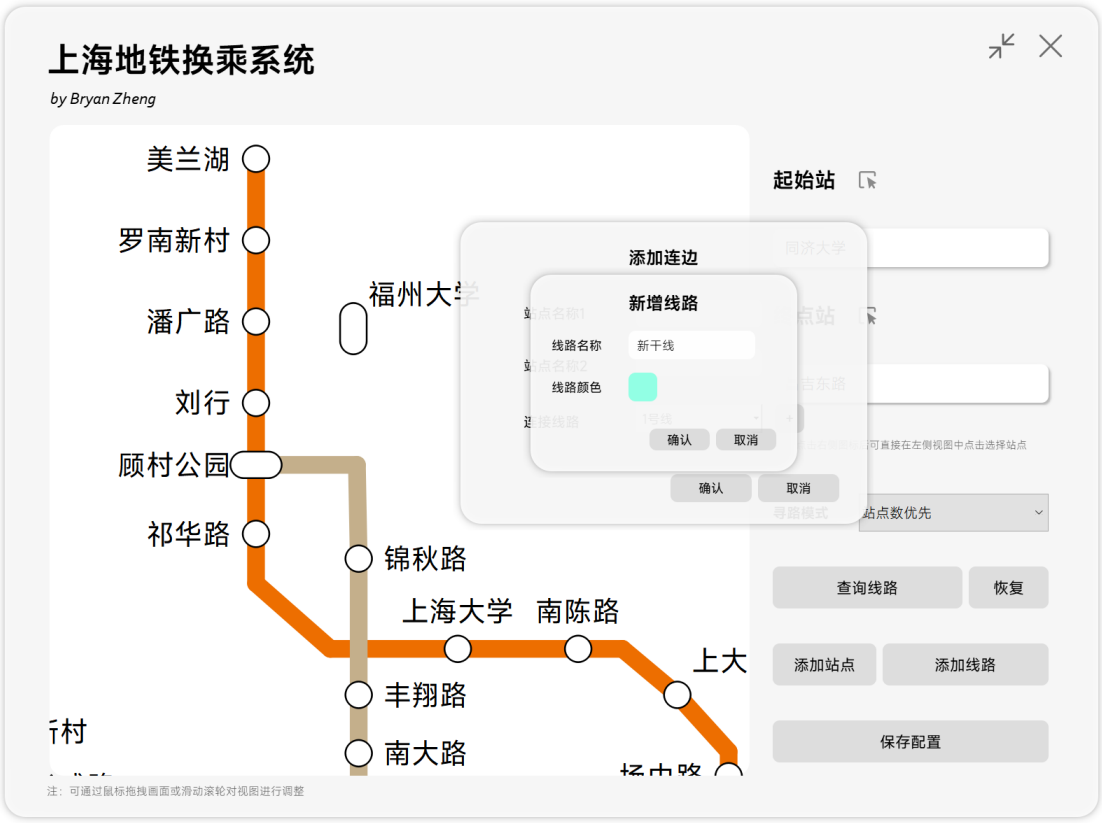
可以点击“+”按钮新建线路：

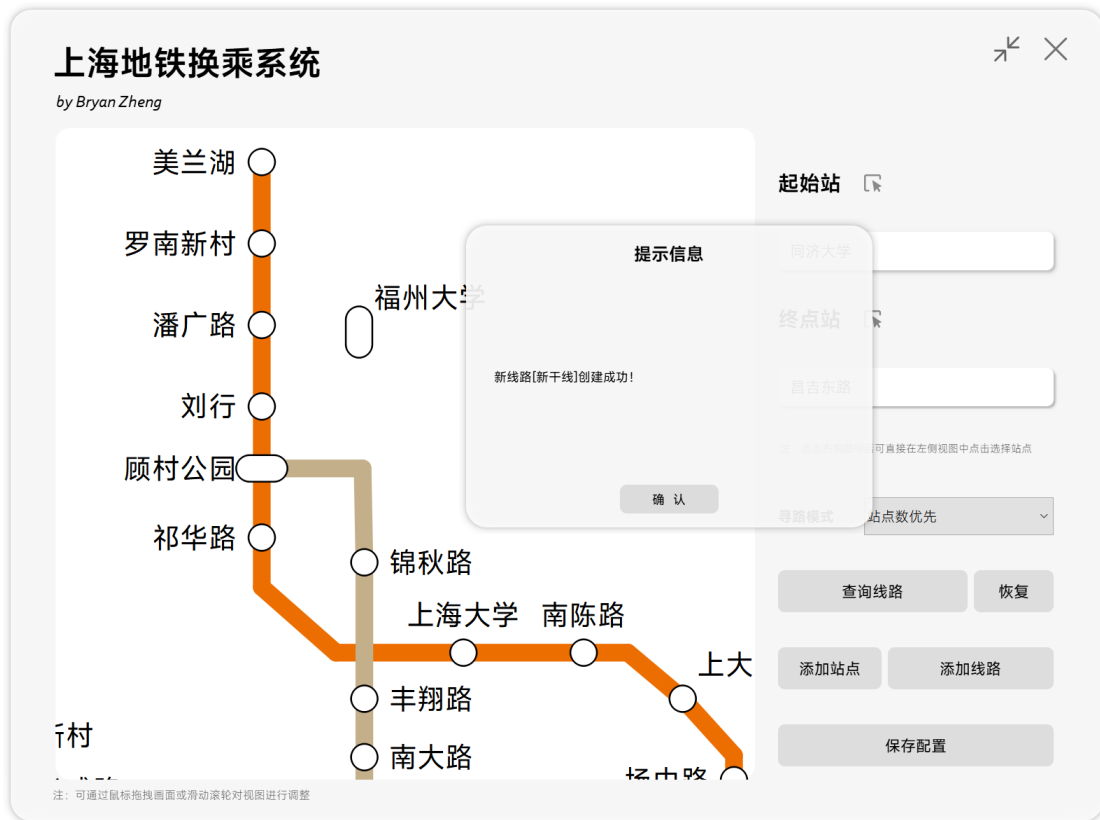


单击线路颜色右侧的颜色方块可以选择线路颜色：

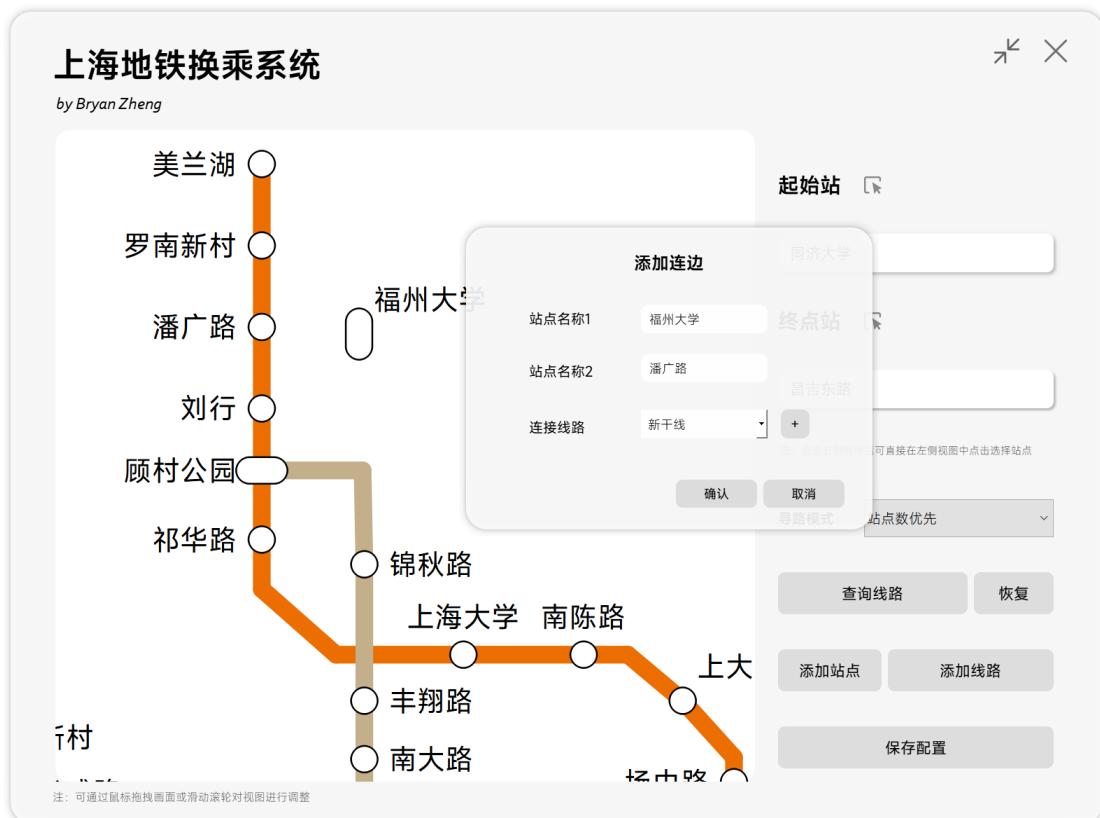


输入线路名称与颜色创建新线路：





输入线路信息，可以选择刚刚新建的线路，点击确认新建成功。





可以对新增加的站点和线路正常寻路：



点击“保存配置”按钮，可以将配置信息保存，下一次打开软件时，所添加的新线路和新站点仍然会被保存。



再次打开软件，配置信息仍然被保存：



第三部分 实践总结

3.1.所做的工作

- **学习掌握并且运用了 Qt5 的编程架构和相关知识：** 我深入学习了 Qt5 框架的编程体系结构，包括窗口、小部件、信号与槽机制、事件处理等。这使我能够构建具有用户友好界面的应用程序，并且有效地处理用户输入和事件。

- **学习并掌握了 Qt5 的对象树与信号槽机制：** Qt5 的对象树和信号槽机制是该框架的核心特性，我深入理解了它们的工作原理并成功应用于项目中。对象树的管理让我能够高效地管理内存资源，而信号槽机制则实现了模块之间的松耦合通信。

- **进一步了解了面向对象的优点并且在此次实践中灵活运用：** 面向对象编程的优点包括封装、继承和多态，我在这次项目中灵活应用了这些概念，将代码组织得更清晰、可维护性更高。

- **复习了二叉树的前序、中序与后序遍历及其线索化知识：** 在二叉树遍历项目中，我回顾了二叉树的基本遍历方式，包括前序、中序和后序遍历，并进一步学习了二叉树线索化的知识，将其应用于项目中，实现了线索二叉树的前序遍历。

- **对迪杰斯特拉算法等最短路径算法进行进一步的加深理解：** 在上海地铁项目中，我加深了对最短路径算法的理解，特别是迪杰斯特拉算法。我不仅理论上掌握了这些算法的原理，还成功地将它们实现应用于项目中，实现了地铁站点之间的最短路径搜索。

- **独立完成实践中算法实现题的全部设计、编程、调试等一系列工作：** 在二叉树项目中，我独立完成了线索二叉树的前序遍历算法的设计、编码和测试，确保其正确性和效率。

- **独立完成实践中综合应用题的全部设计、编程、调试等一系列工作：** 在上海地铁项目中，我独立完成了整个软件的设计、编码和测试，包括地铁地图数据的读取、路径搜索算法的实现、可视化展示等各个方面的工作。

- **完成课程设计报告一份：** 最后，我独立完成了课程设计报告的撰写，总结了项目的整体过程、设计思路和所获得的经验与收获。

3.2. 总结与收获

在完成本次数据结构课程设计的二叉树遍历和上海地铁两个课程设计项目的过程中，我积累了丰富的编程和自学经验，获得了不少宝贵的收获：

3.2.1 能力提升

在本次数据结构课程设计的实践过程中，我不仅复习了数据结构理论课中学习到的二叉树前序、中序和后序以及其线索化的算法、图的邻接表存储方式以及 Dijkstra 单源最短路径的算法，还从零基础地开始学习了 Qt 的图形化应用开发^{[13][14]}。在本次课程设计的项目中，我首次接触到了 Qt 框架，学会了如何使用这一图形化编程工具。我独立自主地学习了 Qt 的信号槽机制以及对象树机制，一点一滴地在实践中提升了自己独立完成一个项目的设计、编程与调试能力。此外，这次实践还强化了我独立思考、解决问题和自主学习的能力。

另外，我也学会了在使用新软件时，可以先查阅自带的帮助文档，这一点在解决问题时非常有帮助。在面对诸如 QGraphicsView、QGraphicsItem、QPainter 等陌生的类时，我通过查阅 Qt 的官方文档，了解了它们的方法的使用方式；此外，我还通过官方文档了解了 QComboBox、QCheckBox、QPushButton 对应的 signal 信号，对于我使用信号槽这一机制有很大的帮助。在设计项目构架时，我学到了自顶向下的设计思想，将复杂任务分解为小块并逐步实现，这对于管理大型项目非常重要。同时，我深刻体会到了面向对象编程的优势，如良好的代码结构和封装性，这对于提高代码的可维护性和复用性至关重要。

3.2.2 课程设计的收获

在本次课程设计的编程实践过程中，我属实收获颇丰。通过本次课程设计的代码撰写与程序调试，我不仅加深了技术能力，还在其他方面许多方面都有所收获。

首先，技术方面，对 Qt 编程框架的学习和掌握使我能够更自如地开发图形化应用程序。在此之前，我只学习过通过 html 搭配 JavaScript 的方式来构建图形化界面，以 C++ 和 Qt 构建图形界面的编程方式以前对我来说是陌生的，但通过不断的实践和学习，我现在能够创建更友好、美观和用户友好的应用程序。此外，通过对 Github 开源项目托管平台的应用，我进一步了解了现代软件开发的工作流程，这对我未来的职业发展将大有裨益。

其次，这次课程设计锻炼了我的独立解决问题的能力。在项目开发过程中，我遇到了各种各样的技术难题和 bug，但我坚持不懈地寻找解决方案，通过独立思考和实践，最终找到

了合适的解决办法。这种经验不仅提高了我的编程技能，还增强了我的自信心，让我相信无论遇到多大的挑战，我都能够克服。

另外，课程设计还培养了我的项目工程思维。我学会了如何规划项目、分析需求、设计系统架构，并按计划逐步推进项目的开发。这种思维方式对于日后处理更复杂的项目和工作任务至关重要。此外，我更加注重了代码的可维护性和可扩展性，这有助于确保项目的长期健康发展。

3.2.3 个人体会

在完成本次项目的过程中，我获得了许多宝贵的体会和收获，相信通过本次作业积累的宝贵经验都将对我的未来发展产生深远的影响。

首先，我深刻认识到了自学能力的重要性。尽管我之前已经掌握了一些编程知识，但在面对全新的编程框架和技术时，我需要自主学习和探索。通过查阅资料、学习文档和解决问题，我不仅复习了 C++ 的相应知识，也学会了使用 Qt 进行图形化编程。这个过程锻炼了我的自学能力，让我明白只要有足够的决心和毅力，我可以攻克任何技术难关。

其次，我意识到了项目工程思维的价值。在整个项目周期中，我需要从项目规划、需求分析、架构设计到编码实现，再到测试和维护，这个全面的过程让我更好地理解项目管理和工程化开发的概念。我学会了如何分析需求、合理规划时间、有效地分配任务，这些技能在未来的工作中将非常重要。

另外，我强化了解决问题与时间管理的能力。在项目开发中，我面临了各种技术挑战和难题。但通过不断的思考和实践，我学会了如何冷静应对问题、找到解决方案。这种解决问题的经验不仅提高了我的编程技能，还培养了我的毅力和耐心，我相信这些品质在未来的工作和生活中将起到关键作用。此外，在本次项目的开发实践过程中，我学会了如何高效地分配时间，合理安排任务，以确保项目按计划进行。

总的来说，这个项目不仅让我获得了技术上的提升，还培养了我的自学、项目管理、问题解决和时间管理能力。希望在本次课程设计中获得的这些经验能够成为我未来发展的坚实基础，让我更加自信地面对各种挑战和机遇！

第四部分 参考文献

- [1] 刘华敏.栈在二叉树遍历中的应用[J].中国多媒体与网络教学学报(上旬刊),2018(03):5-6.
- [2] 郭小春,王红.线索二叉树算法的实验与实现[J].泰山学院学报,2011,33(06):40-45.
- [3] 周敏,于瀛军.浅谈二叉树线索化及利用线索进行遍历[J].农业网络信息,2006(08):33-35.
- [4] 杨晓波,陈邦泽.线索二叉树的可视化实现[J].西北师范大学学报(自然科学版),2013,49(01):46-49.DOI:10.16783/j.cnki.nwnuz.2013.01.010.
- [5] Stanley B. Lippman, Josee Lajoie, Barbara E. Moo, C++ Primer 中文版, 电子工业出版社, 2013
- [6] 范俊怡,刘栩含,龙玲.关于信息学竞赛中最短路算法的研究[J].科技与创新,2021(16):56-58.DOI:10.15913/j.cnki.kjyex.2021.16.024.
- [7] 曾庆红,杨桥艳.最短路问题算法综述[J].保山学院学报,2019,38(05):44-46.
- [8] Pan L,Wulan H. A graph algorithm for the time constrained shortest path[J]. Connection Science,2022,34(1).
- [9] 陈东银,宋艳敏. Dijkstra 算法在地铁换乘中的应用[J]. 信息系统工程, 2012 (10): 83-84.
- [10] 杜轶波. 铁路车站站场图模型及其数据结构选择[J]. 控制工程, 2017, 24(10): 1975-1980.
- [11] 贾敬霞,王振宇,何孝敏等.基于 C 语言的迪杰斯特拉算法仿真与实现[J].电脑编程技巧与维护,2020(03):9-11.DOI:10.16184/j.cnki.comprg.2020.03.003.
- [12] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms Third Edition, 机械工业出版社, 2013
- [13] 霍亚飞.Qt Creator 快速入门第二版[M].北京:北京航空航天大学出版社,2014-1
- [14] 霍亚飞,程梁.QT5 编程入门[M].北京:北京航空航天大学出版社,2015-1